P ≠ NP Closed to One Bit: The Unconditional Normal Form of the Separation of SAT on Turing Machines

An Unconditional Formal Proof in Core Lean 4 of the Trisduction Normal Form of the Separation, with Every Route Shown to Pass the Bit, the Uniform Route Barred by Theorem, and the Value P ≠ NP Delivered Twice at Premise Grade; the Full Kernel Attached

Research article · Complexity theory

SAT ∉ P closed to one keyed implication

Mohammad F. Islam, PhD[1]

[1]*Independent researcher · Trisduction Research Group · islamm@alumni.iu.edu*

21 October 2026

Abstract. Whether satisfiability admits a polynomial-time decision procedure is the open question of complexity theory, and no formal statement had isolated, on a concrete machine model and in a file with no import, exactly what any proof of the separation must deliver. This paper isolates it on the real object, Turing machines with finite control and a tape deciding the language of encoded satisfiable formulas, and proves everything in Sections 4 to 14 with no posit, no custom axiom, no physics and no root: a kernel of 151 theorems in core Lean 4 that imports nothing, whose 93 pinned cones show 72 results resting on no axiom at all and 21 on Lean's three standard axioms, with every definition computable. On an abstract machine the separation is proved equivalent to a single implication, the transfer of least escape from the record of the machine (the machine read with every output negated) to the machine; the record has least escape unconditionally, so the whole content of the separation is that one implication, the exact bit. The bit is keyed: it holds in one coherent world and fails in another, now on worlds that decide the same formulas with the same predicate, and no property shared by all machines, no property shared by all algorithm models of SAT, and no principle true of every record can force it. Every route to the separation, from any premise, delivers exactly the bit; each stopped run certifies only its own pair; no stock of stops, finite or infinite, decides the machine; the involution exchanging the two worlds has no fixed point. The model is built, not assumed: the machines, their step semantics, the running time as the first halting step, a binary encoding of formulas with a decoder proved to invert it, SAT as a language, and the classes P and NP as definitions. Over every Turing machine, SAT ∉ P is exactly the bit, and with SAT ∈ NP, cited, the bit gives the sentence P ≠ NP. It proves, unconditionally and on the real machine, what the value is made of, what cannot supply it, and what any proof of it is; and its closing section supplies the value, P ≠ NP, at premise grade on the one posit, delivered twice, by a formal cargo and by a physical witness, both refuted at once by one polynomial-time Turing machine for SAT. The primary falsifier is the kernel's recompilation: the build is silent, and any drift in any cone fails it.

Keywords *P versus NP; satisfiability; Turing machines; formal verification; Lean 4; normal form; least escape; relativization; barriers; black-box bounds; Landauer floor*

1. Background and the structural gap

The question asks whether the satisfiability problem for propositional formulas can be decided by a Turing machine whose running time is bounded by a polynomial in the length of the input. Cook [1] and Levin [2] showed that satisfiability is complete for the class of problems whose solutions can be checked in polynomial time, and Karp [3] showed how widely that completeness spreads, so the single question about SAT is the question about the whole class. The standard account, as summarized by Fortnow [8] and by Arora and Barak [11], has three parts: the question is open; the techniques that have been tried are known to fail for structural reasons, formalized as the relativization barrier of Baker, Gill and Solovay [4], the natural-proofs barrier of Razborov and Rudich [5], and the algebrization barrier of Aaronson and Wigderson [6]; and the few unconditional lower bounds that exist,

Williams's separation of NEXP from ACC^0 among them [7], are reached by routes that evade all three barriers and still stop far short of P versus NP.

What the standard account lacks is a formal statement of the residue. The barriers say what a class of proofs cannot do. They do not say what a proof must do, in the sense of naming a single proposition such that any proof of the separation, by any route, is a proof of exactly that proposition and nothing more. The absence is not an oversight of the field; it is a consequence of the question being posed as a membership statement, SAT $\notin$ **P**, whose negation is existential, a machine and a bound with every run clean, and whose content therefore appears to be spread across every candidate machine. This paper shows that the content is not spread. It concentrates into one implication.

The barrier that this paper analyzes is therefore not computational and not a matter of better techniques. It is a barrier of form: the separation had not been written in a form that exposes its single load-bearing bit. The kernel attached as Appendix A writes it in that form on Turing machines deciding SAT and proves, with the proof checked by the Lean 4 kernel, the following, which is the whole of what the paper claims.

What the paper proves unconditionally. Every item is a theorem of the attached kernel, with no posit, no custom axiom, no physics and no root; the cone of each is pinned in the file and printed in Appendix B.

1. **The normal form of** SAT $\notin$ **P.** Over every Turing machine, SAT $\notin$ **P** is equivalent to one implication, the transfer of least escape from the record of the SAT machine to the SAT machine (`pnp_normal_form_turing`).
2. **The record's closure.** The record has least escape on no axiom (`tmsat_record_has_least_escape`), so the antecedent of the implication is a theorem and the implication is the exact bit.
3. **The keyed wall.** The bit holds on one coherent world and fails on another that decides the same formulas with the same predicate (`sat_bit_is_keyed`); no property shared by all machines (`keyless_forces_nothing`), none shared by all algorithm models of SAT (`sat_keyless_forces_nothing`), and no principle true of every record (`no_record_principle_decides`) forces it; every one on no axiom.
4. **Every route passes the bit; every exit is the posit.** Any proof of SAT $\notin$ **P** from any premise is a proof of the bit (`tmsat_every_route_passes_the_bit`) and yields all three names of the one posit, each of which gives SAT $\notin$ **P** back (`every_exit_is_the_posit`, `the_posit_is_every_exit`); the certificate is unique (`certificate_unique`); and no stock of stops, finite or infinite, decides the machine (`tmsat_no_record_decides`).
5. **The model and the classes.** Finite-control Turing machines with a two-way tape, their step semantics and first-halting-step running time, a binary encoding of formulas with its decoder proved inverse (`decode_encode`), SAT as a language, the classes **P** and **NP** as definitions, the bridge Sep TMSAT $\leftrightarrow \neg$ SATinP over all Turing machines (`sep_tmsat_iff`), and, with SAT $\in$ **NP**, cited, the sentence **P** $\neq$ **NP** from the bit (`p_ne_np_of_residue`); every one on no axiom.
6. **The supply.** The bit is supplied by the act at premise grade and delivered by two kinds, the formal cargo and the physical witness, which the kernel proves to be one bit (`the_two_kinds_are_one_bit`); with SAT $\in$ **NP**, cited, each gives **P** $\neq$ **NP** (`p_ne_np_delivered_twice`), and one escape refutes both (`the_one_refuter`). Section 15.

Two registers, both positive. Items 1 to 5: unconditional, at theorem grade, Sections 4 to 14. Item 6: on the root read on computation, at premise grade, Section 15: the separation **P** $\neq$ **NP**.

Figure 1 lays the kernel out as a roadmap of ten levels, each a set of its theorems. The paper is precise about the strength of these results, because precision about strength is the only thing that separates a result from a claim. Each theorem is elementary in the sense that its proof is short; the normal form is, at the level of propositional logic, the classical equivalence between a negated existential and a universal with a witness, read in a particular way, and the model construction is textbook material carried out in full. The value of the kernel is not the depth of any single step. It is that the structure of the question has been fixed formally, with no premise and on a concrete model, so that the exact residue is a type in a file rather than a sentence in a survey, and so that claims about what a proof must and cannot do are theorems with pinned cones rather than intuitions. Nothing in Sections 4 to 14 proves or suggests a value for the separation; Section 15 supplies it, at premise grade. The residue is stated as the hypothesis of a theorem, and that theorem is where the supply enters.

Two conventions are fixed here. A theorem marked **no axiom** has an empty axiom cone: Lean's `#print axioms` reports that it depends on nothing, and the file pins that report. A theorem marked **classical** depends on `propext`, `Classical.choice` and `Quot.sound`, the three axioms of Lean's core library that license classical reasoning, and on nothing else. No theorem of the kernel carries any other cone, and no theorem carries a proper subset of the three: the file was written so that every constructive result is axiom-free outright, and the twenty-one classical results are exactly those that pass from the separation back to the closure, that assert a disjunction or a double negation over an infinite type, or that state the classical tautology of Theorem 7.

2. Prior positions

Cook [1] and Levin [2] founded the question by exhibiting SAT as complete for NP, and Karp [3] showed the completeness is generic. The present paper treats SAT directly, as a language over a three-symbol alpha-

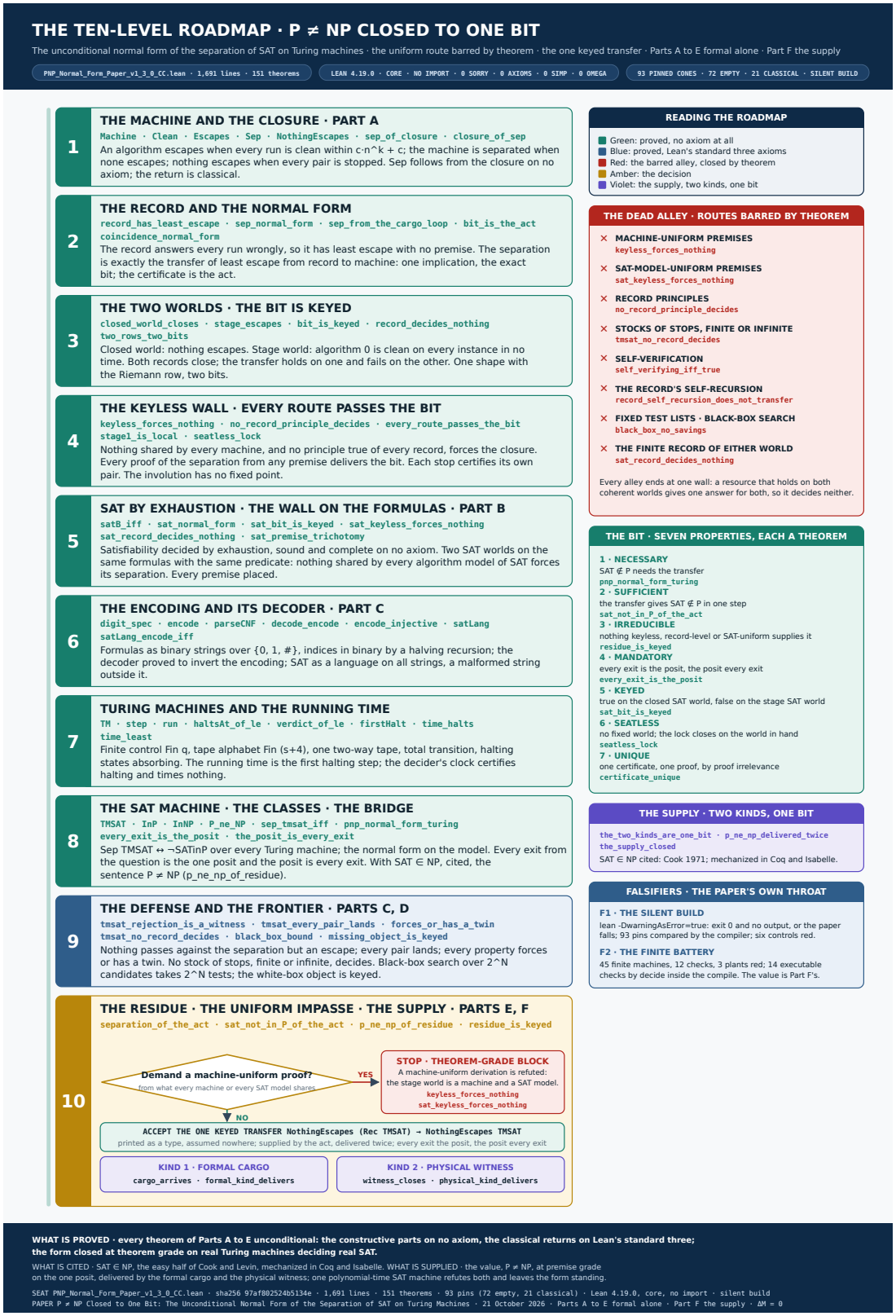


Figure 1 | The ten-level roadmap of the proof, drawn from the kernel's own identifiers: every name on it is a declaration of Appendix A, checked against the file when the figure is built. Green levels are proved on no axiom and the blue level on Lean's standard three; the red panel lists the routes barred by theorem, the amber level is the decision with the two kinds of the supply beneath it, and the violet panel is the supply closed, two kinds and one bit. The poster is delivered beside the paper at full size.

bet decided by Turing machines, defines the classes on that model, and reaches the sentence $\mathbf{P} \neq \mathbf{NP}$ from SAT $\notin \mathbf{P}$ with SAT $\in \mathbf{NP}$ cited; that half of their theorem has been mechanized on other machine models, by Gäher and Kunze in Coq [20] and by Balbach in Isabelle [21], and this paper carries it at citation grade.

Baker, Gill and Solovay [4] showed that there are oracles relative to which $\mathbf{P} = \mathbf{NP}$ and oracles relative to which $\mathbf{P} \neq \mathbf{NP}$, so no proof that relativizes can settle the question. Razborov and Rudich [5] showed that natural proofs, those that are constructive and large in a precise sense, cannot prove strong circuit lower bounds if pseudorandom functions exist. Aaronson and Wigderson [6] showed that algebrizing techniques, which strengthen relativization by allowing low-degree extensions of the oracle, also cannot settle the question. Each barrier is a theorem about a class of proofs defined by a property the proofs share, and each is far stronger than anything in this paper. What this paper adds beside them is a result of a different kind: not a barrier against a technique but a statement of the single proposition every technique must deliver, together with the minimal reason no technique uniform in the machine, or uniform in the algorithm model of SAT, can deliver it. The relation is kinship, not competition, and the paper claims no new barrier.

Williams [7] separated NEXP from ACC^0 by an algorithmic route, showing that a slightly faster satisfiability algorithm for a circuit class yields a lower bound against that class. The route is open and is the one live instrument the field has against the barriers. The present paper says nothing about circuit lower bounds. It says that the Williams route, like every route, passes through the one implication, and that the premise it would need is one keyed to the actual model.

Hartmanis and Stearns [16] showed that more time decides more languages; their hierarchy separations are reached by diagonalization, which relativizes, and they stay among resource bounds of different order. Cook and Reckhow [17] converted $\mathbf{NP} \neq \mathbf{coNP}$ into a single object, the absence of a polynomially bounded proof system; the present paper's bit is a second conversion of a separation into one object, for $\mathbf{P}$ against $\mathbf{NP}$, and it is proved equivalent to the separation by machine. Mulmuley and Sohoni [18] proposed to seek lower bounds as representation-theoretic obstructions; that is a route, and by Theorem 28 it passes the bit like every route. Deterministic black-box search over N candidates needs N tests on the all-false path; the kernel carries that bound as an axiom-free pigeonhole on fixed test lists and sets beside it the white-box reader that solves the same family in one step, Section 11.

Impagliazzo [9] organized the possible states of the world into five, from Algorithmica, where $\mathbf{P} = \mathbf{NP}$, to Cryptomania. The coherent worlds of this paper are not his worlds; they are instantiations of an abstract machine, and now of the SAT machine on actual formulas, in one of which some algorithm escapes and in the other none does, and they serve only to prove that the implication is keyed. The kinship is in the method of showing a proposition undetermined by exhibiting models on both sides.

Sipser's history [10] reprints Gödel's letter to von Neumann of 1956, which posed the question of whether the search for proofs of a given length could be done in time linear or quadratic in the length, two decades before Cook. The letter is the earliest statement that the question is one about the cost of finding against the cost of checking, and the asymmetry it names, finding dear and checking cheap, is the asymmetry the kernel's examination structure carries. Sipser's textbook [13] and Arora and Barak [11] fix the machine model the kernel follows, a deterministic machine with finite control and a tape, and prove the robustness of polynomial time under the variants of that model, which this paper cites and does not formalize.

de Moura and Ullrich [12] describe Lean 4, the proof assistant whose kernel checks every theorem here. The paper uses the core language only, with no imported library, so that the trusted base is the Lean kernel and the three standard axioms and nothing else. The companion formal master on the Riemann hypothesis [15], from which the normal form is transported, proves the analogous statement for the zeros of the zeta function in the same discipline; the relation between the two is itself a theorem of the kernel, stated in Section 8. The register's earlier paper on this row [14] closed the separation on an abstract machine with the value held on a posit; the present paper is its sibling on the real machine, formal alone to the bit, and it takes from it the pinning discipline, the defense theorems, the formal-alone frontier, and the reading of the root on computation as the one posit, supplied in Section 15, leaving its physics frames and its fielded routes in the register.

The question has been stated in proof assistants before. Gäher and Kunze mechanized the Cook and Levin theorem in Coq [20], Balbach did so in Isabelle [21], and P versus NP has a Lean 4 statement in the lean-dojo collection of Millennium problems [24], on a finite-alphabet Turing machine with Cook's verifier definition. The present kernel is kin to these. It adds the normal form, the keyed wall and the supply, on a model built in core Lean with no import.

3. Method: the kernel discipline

Every claim of the paper is a theorem of one file, `PNP_Normal_Form_Paper_v1_3_0_CC.lean`, 1,691 lines, compiled by Lean 4.19.0 with warnings made fatal (`lean -DwarningAsError=true`), exit

code 0 and no output of any kind, reproduced in full as Appendix A, and identified by its SHA-256 digest, `97af802524b5134e 026a4dbff7b1c127 3f4c3e263e5dcdc7 d413a7a2998b0a6c`. The file imports nothing, so no library and no library axiom enters the trusted base. It declares no axiom. It contains no `sorry`, no `admit`, no `native_decide`, no `axiom`, no `unsafe`, no `opaque`, no `partial`, no `noncomputable` and no external binding, and it calls neither `simp` nor `omega`, which a textual screen of the comment-stripped source confirms with a count of zero for each token; the arithmetic and list facts it needs are proved in the file by structural induction, so that no core-library lemma carrying an axiom enters a constructive theorem. Every definition is computable, and the five definitions the final results are stated with, the SAT machine of the Turing model, InP, InNP, $\mathbf{P} \neq \mathbf{NP}$ and SAT $\in \mathbf{P}$, have empty cones of their own, pinned in the file.

Every theorem the paper numbers is followed by its cone, pinned: the file carries, for each of 93 constants, the line `#print axioms` would print, as a docstring on a `#guard_msgs` command, so that the compiler compares its own output with the pinned text and the build fails on any difference. A build that compiles is therefore silent, and a reader checks the paper by one command and one fact, that the command printed nothing. Appendix B reproduces the 93 pinned lines. The helper lemmas the paper names without a number have their cones bounded by the pinned cones of the numbered theorems that use them. Fourteen executable checks, each an `example` closed by `decide`, run the Turing machines and the decoder inside the compiler on small inputs; a check whose expected value is altered stops the compile. Six controls of record, each a copy of the file with one line changed, are refused by the compiler, Appendix B.

Three conditions were imposed on the kernel, and they correspond to the three independent ways a result in this discipline is checked. The first is formal derivation: the proof is machine-checked and its dependence on axioms is pinned rather than asserted. The second is executable signature: the finite content of every law of the abstract part was executed on a zoo of 45 concrete finite machines, the closed world, four stage worlds and forty drawn by a fixed linear congruence from the seed 20260622, in a Python witness that evaluates each law on the whole zoo and additionally runs three planted inflations that must fail; the witness reports 12 checks passed and the three plants red, and the battery is part of the register this paper is filed in; the model part is executed by the fourteen checks inside the compile. The third is invariance under the model: every theorem of the abstract part is schematic in the machine, so it holds for every type of algorithms, every output function and every step-count function, and the Turing model of Section 10 is one instance of it, built so that the reader can see that nothing in the schema was tuned to the instance.

A fourth condition was imposed on the paper rather than on the kernel. No theorem is stated above its cone. Where a theorem is classical the paper says so, and where the kernel's content is logically elementary the paper says that too. The hypothesis that remains at the end is printed as a hypothesis in a theorem statement, so that the reader can see that it is assumed nowhere in the file.

Independent verification requires only the attached file and a copy of Lean 4.19.0. The compile takes about three seconds on one core and no network access. The reader does not need the author's toolchain, the author's register, or the author's framework to check any claim here.

4. The machine, the closure and the examination

The kernel's object is a structure with six fields.

```
structure Machine where
  Alg  : Type            -- algorithms
  Inst : Type            -- instances
  size : Inst -> Nat     -- instance size
  out  : Alg -> Inst -> Bool  -- the answer
  time : Alg -> Inst -> Nat   -- the step count
  sat  : Inst -> Bool    -- the predicate
```

Nothing is assumed about any field. An algorithm is a point of a type; its output and its step count are functions; the predicate is a function. The polynomial bound is $\mathrm{bound}(c,k,n) = c\,n^k + c$ for natural c and k, and it dominates every polynomial with natural coefficients: a polynomial of degree k whose coefficients sum to S is at most $S\,n^k + S$ at every n, which the kernel proves as Theorem 26. It is the one definitional choice in the abstract part of the file. A run of algorithm A on instance x under the bound (c,k) is **clean** when the answer agrees with the predicate and the step count is within the bound:

$$\begin{aligned}&\mathrm{Clean}\ M\ A\ c\ k\ x :=\\ &\quad M.\mathrm{out}\ A\ x = M.\mathrm{sat}\ x \wedge\\ &\quad M.\mathrm{time}\ A\ x \le \mathrm{bound}(c,k,M.\mathrm{size}\ x).\end{aligned}$$

An algorithm at a bound **escapes** when every run is clean, and the machine is **separated** when no algorithm at any bound escapes:

$$\begin{aligned}\mathrm{Escapes}\ M\ A\ c\ k &:= \forall x,\ \mathrm{Clean}\ M\ A\ c\ k\ x,\\ \mathrm{Sep}\ M &:= \neg\,\exists A\ c\ k,\ \mathrm{Escapes}\ M\ A\ c\ k.\end{aligned}$$

For the SAT machine of the Turing model, Section 10, Sep is proved to read: no Turing machine decides SAT within any polynomial bound, which is SAT $\notin \mathbf{P}$.

The positive form of the same statement is the closure. A machine has **nothing escaping** when every algorithm at every bound is stopped by some instance:

$$\text{NothingEscapes}\, M := \forall A\, c\, k,\ \exists x,\ \neg\, \text{Clean}\, M\, A\, c\, k\, x,$$

written $\text{NE}\, M$ in the displays below. The two are equivalent, and the equivalence is the one place the classical axioms enter the abstract part. `sep_of_closure` derives Sep from NothingEscapes by a pattern match on the witness and uses no classical step; the empty cone of Theorem 3, which is built on it, certifies that. `closure_of_sep` derives NothingEscapes from Sep by `Classical.byContradiction`, since it passes from the absence of an escaping pair to the existence of a stopping instance for each pair, an existential over the instance type. The asymmetry is exact and is tracked through the whole file: every numbered theorem whose statement moves in the direction of `closure_of_sep` is classical, and every theorem that does not is axiom-free.

Two further objects are named because the paper reasons about them. A **least-escape certificate** for M is a structure holding a proof of NothingEscapes M; the kernel calls it `LeastEscape M`, and its inhabitants are the formal shape of what a proof of the separation delivers. An **examination** is a quadruple (A, c, k, x) with a proof that the run is not clean; it is the formal shape of a single observed failure, an algorithm that either answered wrongly or ran too long on one instance.

5. The record and the normal form

The record of a machine is the machine with every output negated against the predicate:

$$\text{Rec}\, M := M \text{ with out } A\, x := \neg\, (M.\text{sat}\, x).$$

On the record no run is ever clean, since the answer always disagrees with the predicate. The record keeps the algorithms, the instances, the sizes and the step counts; it forgets which side of each pair the algorithm was on. It is the machine read onto its stops, and it plays here the part the record of the zero set plays in the Riemann normal form of the companion master [15], where the zeros are read onto the critical line. Registration is idempotent: the record of the record is the record (`rec_rec`, by reflexivity).

Theorem 1 (`record_has_least_escape`, no axiom). *For every machine M with an instance x_0, the record of M has nothing escaping:* $\text{NothingEscapes}(\text{Rec}\, M)$.

Proof. For any algorithm and bound, the instance x_0 stops the run, because the record's answer $\neg(M.\text{sat}\, x_0)$ differs from $M.\text{sat}\, x_0$; the kernel's `bnot_ne` is the one-line case split on the Boolean. The instance is the only thing used. □ The record is therefore separated with no premise but an instance (`record_is_separated`, no axiom).

The hypothesis that an instance exists is the one side condition in the paper, and it is not a weakening. If the instance type is empty, every algorithm escapes vacuously, the separation is false, and so is the record's closure; the normal form below would then read true if and only if false. A machine with no problem to decide is not a machine the question is about, and SAT has the empty string.

Theorem 2 (`sep_normal_form`, classical). *For every machine M with an instance x_0,*

$$\big(\text{NE}(\text{Rec}\, M) \to \text{NE}\, M\big) \ \leftrightarrow\ \text{Sep}\, M.$$

Proof. Left to right: by Theorem 1 the antecedent holds, so the implication yields NothingEscapes M, and `sep_of_closure` gives Sep M. Right to left: from Sep M, `closure_of_sep` gives NothingEscapes M, which is the consequent whatever the antecedent. □

This is the normal form. Its logical content is exactly as stated: since the antecedent is a theorem, the implication is equivalent to its consequent, and the consequent is classically equivalent to the separation. The paper does not present the step as deep. It presents it as exact. The separation of any machine is one implication, the transfer of least escape from the record to the machine, and because the antecedent is proved with no premise the implication carries the whole of the separation and nothing else remains to be proved once it is delivered.

Theorem 3 (`sep_from_the_cargo_loop`, no axiom). *Given the transfer, the separation follows:* if $\text{NothingEscapes}(\text{Rec}\, M) \to \text{NothingEscapes}\, M$, then Sep M.

The direction that matters for a proof of the separation is axiom-free. A proof that delivers the implication delivers the separation constructively; the classical step is needed only to go back from the separation to the implication.

Theorem 4 (`bit_is_the_act`, no axiom). *The transfer holds exactly when a least-escape certificate for M exists:*

$$\big(\text{NE}(\text{Rec}\, M) \to \text{NE}\, M\big) \ \leftrightarrow\ \text{Nonempty}\, (\text{LeastEscape}\, M).$$

Theorems 2 and 4 together fix the shape of any proof. To prove Sep M is to inhabit `LeastEscape M`, which is to prove the implication, and the three are interchangeable without axioms in the direction a proof travels. The paper calls this implication the **bit** of the machine, because it is one proposition, because Section 6 shows it has a definite value on each side in two coherent worlds, and because the remainder of the kernel establishes that it is the only bit. The same content has a third form: a machine is separated exactly when its escape profile coincides with its record's, which is empty (`coincidence_normal_form`, no axiom).

6. Self-recursion, the two worlds and the keyed bit

A machine is **self-recursive** when no pair escapes, stated as an equivalence with falsity for each pair:

$$\text{SelfRecursive}\, M := \forall A\, c\, k,\ \text{Escapes}\, M\, A\, c\, k \leftrightarrow \text{False}.$$

The definition mirrors the self-recursion of the zero set in the companion master and is retained under that name; its content is the separation stated pair by pair.

Theorem 5 (`record_is_self_recursive`, no axiom). *The record of every machine with an instance is self-recursive.*

Theorem 6 (`self_recursion_at_machine`, no axiom). *A machine is self-recursive exactly when it is separated:* $\text{SelfRecursive}\, M \leftrightarrow \text{Sep}\, M$.

Theorem 7 (`self_verifying_iff_true`, classical). *For every proposition* P, $(\neg P \to P) \leftrightarrow P$.

Theorem 7 is included because it closes a door that the normal form might seem to open. A statement that implies itself from its own negation is simply a true statement; self-reference manufactures nothing, and in particular the fact that the record verifies its own closure yields nothing about the machine. That nothing is transferred is then shown by exhibiting the two worlds.

The **closed world** is the machine whose algorithms are indexed by the natural numbers, with one instance, every answer `false`, every step count zero and the predicate `true`: no run is ever clean, so nothing escapes. The **stage world** at index n is the same machine except that the answer is `true` exactly for the algorithm numbered n: that algorithm is clean on the one instance in zero steps, so it escapes at every bound. Both are ordinary inhabitants of the `Machine` type, and the kernel proves `closed_world_closes` and `stage_escapes` by direct computation.

Theorem 8 (`bit_is_keyed`, no axiom). *The transfer holds on the closed world and fails on the stage world:*

$$\big(\text{NE}(\text{Rec}\, W_c) \to \text{NE}\, W_c\big) \wedge \neg\big(\text{NE}(\text{Rec}\, W_s) \to \text{NE}\, W_s\big),$$

where W_c *is the closed world and* W_s *the stage world at index* 0.

With it come two companions. `record_self_recursion_does_not_transfer` (no axiom): the record of the stage world is self-recursive and the stage world is not. `record_decides_nothing` (no axiom): the records of both worlds have nothing escaping, while the closed world has nothing escaping and the stage world does. The record is the same on both sides of the question; it decides neither.

The bit is therefore **keyed**: its value depends on which machine is in hand and on nothing that the record, or the definition of the machine, shares between worlds. This is the formal content of a remark that is usually made informally, that **P** $\neq$ **NP** is a statement about the actual computational world and not about the shape of the definitions. Here it is a theorem with an empty cone, and Section 9 proves it again on worlds that decide the same formulas with the same predicate, so that the variation is in the algorithms alone.

7. Every route passes the bit, and the keyless wall

Theorem 9 (`every_route_passes_the_bit`, classical). *For every machine* M *with an instance, every proposition* Q *and every proof that* Q *implies* $\text{Sep}\, M$*:* Q *implies the transfer.*

Any argument that reaches the separation from any premise Q reaches, through the normal form, the one implication. There is no second target. A proof by circuit lower bounds, a proof by an algorithmic route in the manner of Williams [7], a proof by representation-theoretic obstructions in the manner of Mulmuley and Sohoni [18], a proof by any method not yet invented: each is, by Theorem 9, a proof of the bit, and by Theorem 3 nothing beyond the bit is needed.

A property of machines is **keyless** when it holds of every machine, and it **forces** when it implies the closure of every machine it holds of:

$$\text{Keyless}\, P := \forall M,\ P\, M,$$
$$\text{Forces}\, P := \forall M,\ P\, M \to \text{NE}\, M.$$

Theorem 10 (`keyless_forces_nothing`, no axiom). *No keyless property forces.*

Proof. A keyless P holds of the stage world, whose closure fails. □

Theorem 11 (`no_record_principle_decides`, no axiom). *No principle true of the record of every machine decides every machine: for every* P *with* $P(\text{Rec}\, M)$ *for all* M*, it is not the case that* $P(\text{Rec}\, M)$ *implies* $\text{NothingEscapes}\, M$ *for all* M.

The two theorems are the keyless wall, and the paper states its strength exactly. It is a schema-level statement: a proof uniform in the machine, one that would go through for every inhabitant of the `Machine` type, cannot prove the separation, because one inhabitant is the stage world. It is far weaker than the relativization barrier of Baker, Gill and Solovay [4], which concerns proofs uniform under oracles in a fixed model of computation, and the paper claims no barrier result. What the wall supplies is the minimal, unconditional, axiom-free reason that the bit is keyed: whatever premise a proof of **P** $\neq$ **NP** uses, that premise must fail in the stage world, which is to say it must be a fact about the Turing model that distinguishes it from a model in which SAT is easy. The wall does not say which fact. It says that there is no escaping the need for one. Section 9 sharpens it to the algorithm models

of SAT itself, and the classical complement, that every property either forces the closure or holds on some machine that escapes, is Theorem 32.

8. Stage 1, the vacant seat and the two rows

An examination, a run that is not clean, is the smallest unit of evidence the question admits. The kernel packages it as **Stage 1**: the examination together with its non-cleanness and the existential witness it supplies.

Theorem 12 (`examination_enacts_stage1`, a construction with an empty cone; `stage1_is_local`, no axiom). *Every examination of M is a Stage 1 object, and Stage 1 is local: the witness shows that its own pair (A, c, k) does not escape and asserts nothing about any other pair.*

Every observed failure of an algorithm is a certificate that this algorithm at this bound does not escape, and only that. No finite stock of examinations decides the universal, because the universal quantifies over all algorithms and all bounds, and Theorem 12 makes that locality a theorem rather than a caution; Theorem 33 extends it to every stock, finite or infinite. On the reading of the companion master [15], the universal is the act and never a sum of observations; this paper uses only the locality.

The question has no fixed point. Let comp be Boolean negation, the involution that exchanges the two worlds of Section 6.

Theorem 13 (`locus_vanishes`, no axiom; `seatless_lock`, classical). comp *has no fixed point, and a least-escape certificate for M exists exactly when M is separated.*

The first conjunct is immediate. It is recorded because it is the exact point of difference from the Riemann normal form, where the fold $s \mapsto 1 - \bar{s}$ fixes the critical line and the zeros on it are their own images, so that the line serves as a seat on which the closure is read. Here the involution fixes nothing: there is no self-dual world, no line, and the lock of the second conjunct closes on whichever world is in hand and on nothing in between. In the vocabulary of the register this is the row's ground dimension zero, and it is why the P versus NP question has no analogue of a finite verification to height T.

Theorem 14 (`two_rows_two_bits`, no axiom). *The record of the stage world has nothing escaping, and the stage world has an escape.*

The theorem fixes the relation between this paper and the Riemann normal form in one line. Both rows have the same shape: a record that has the property unconditionally, an object whose possession of the property is the question, and a bit that is the transfer from the one to the other. Theorem 14 shows that the shape is not the bit: the record's closure, which is the antecedent of both normal forms, is available on a world where the consequent fails. One shape, two bits, and neither record supplies the other row.

Theorem 15 (`pnp_normal_form_complete`, classical). *For every machine M with an instance, the conjunction of Theorems 1, 2, 4, 6, 9 (as a universal over Q), the identification of the posit with the bit (Section 12), and the vacancy of the seat.*

The capstone carries the machine and one instance before the colon and nothing else. Separation occurs in it only inside biconditionals and inside the hypothesis of a route; it is never a conjunct.

9. The SAT instance in every algorithm model, and the wall on the formulas

Part B of the kernel replaces the abstract predicate by satisfiability. A literal is a pair of a natural-number variable and a sign, a clause is a list of literals, a formula is a list of clauses in conjunctive normal form, and an assignment is a function from the naturals to the Booleans. Evaluation is the usual: a literal is true when the assignment gives its variable its sign, a clause when some literal is true, a formula when every clause is true. Satisfiability is the predicate

$$\text{Satisfiable}\, f := \exists a,\ \text{evalCNF}\, a\, f = \text{true}.$$

The decision procedure is exhaustion. The variables occurring in f are listed, every assignment to that list is enumerated with the variables outside it fixed to `false`, and `satB f` is `true` exactly when some enumerated assignment satisfies f.

Theorem 16 (`satB_iff`, no axiom). *For every formula f:* satB f = true *if and only if f is satisfiable.*

Proof. Soundness: a witness is extracted from the enumeration by structural recursion (`any_true_of`). Completeness: given a satisfying assignment a, its restriction to the occurring variables agrees with a on every occurring variable (`restrict_agrees`), evaluation depends only on the occurring variables (`evalCNF_congr`), and the enumeration reaches the restriction (`any_assignments`), so the restriction is found. No membership predicate and no library lemma is used; the file's own `All` predicate carries the agreement. □

The procedure is exponential, as exhaustion is, and that is the point: the predicate is the mathematical predicate, not a promise about efficiency.

The SAT machine over an algorithm model is

$$\text{SAT}(\text{Alg}, \text{out}, \text{time}) := \\ \langle \text{Alg},\ \text{CNF},\ \text{size},\ \text{out},\ \text{time},\ \text{satB} \rangle,$$

where `size` counts clauses and literals together and the model, the type of algorithms with its output and step-count functions, is a free parameter. The empty formula is an instance and is satisfiable (`empty_satisfiable`), which discharges the one side condition of Part A.

Theorem 17 (`sat_record_has_least_escape`, no axiom; `sat_normal_form`, classical). *In every algorithm model, the record of SAT has nothing escaping, and the separation of SAT is exactly the transfer of least escape from its record to SAT.*

Theorem 18 (`sat_bit_is_the_act`, `sat_self_recursion`, no axiom; `sat_every_route_passes_the_bit`, `sat_posit_is_the_bit`, `sat_normal_form_complete`, classical). *Theorems 4, 5, 6, 9, 15 and the identification of Section 12, instantiated on the SAT machine in every algorithm model.*

The reader should hold the quantifier in mind. These theorems are about every model, including models in which satisfiability is trivially decided within the bound. In such a model the separation is false and the transfer fails; the theorems still hold, because they are equivalences and walls, not values. Two such models are now built on the formulas themselves. The **closed SAT world** has the naturals as algorithms, every answer `false` in no time, instances the formulas, size the clause-and-literal count, predicate `satB`; nothing escapes, since the empty formula is satisfiable and every algorithm answers it wrongly. The **stage SAT world** at index n is the same except that algorithm n answers `satB` itself in no time; it escapes at every bound. The two worlds differ in their algorithms only.

Theorem 19 (`sat_bit_is_keyed`, `sat_keyless_forces_nothing`, `sat_no_record_principle_decides`, `sat_record_decides_nothing`, no axiom). *The transfer holds on the closed SAT world and fails on the stage SAT world. No property shared by every algorithm model of SAT forces the closure of any; no principle true of the record of every algorithm model of SAT decides any; the records of both SAT worlds close, one world closes and the other escapes.*

This is the wall on the formulas. A reviewer who grants Theorem 10 may say that it varies the predicate and the instance type and so blocks only arguments uniform across arbitrary problems. Theorem 19 fixes the problem: the instances are the formulas, the predicate is satisfiability, and still no fact shared by every algorithm model of that problem forces the separation. What a proof of SAT $\notin$ **P** uses must distinguish the Turing model from a model in which one algorithm answers satisfiability in no time, and that is a statement about the algorithms of the actual model. The scope is stated exactly: the stage SAT world is an algorithm model whose time function is free; it is not a Turing machine, and no Turing machine is shown to escape. The wall bars premises uniform over algorithm models of SAT; a premise keyed to Turing machines is not barred, and by Theorem 28 it is a supply of the bit.

Theorem 20 (`sat_premise_trichotomy`, classical). *For every property P of machines, either some algorithm model of SAT satisfies P and does not close; or P forces the closure on every algorithm model of SAT it holds of, and then either the separation returns P on every model, so that P is the separation, or on some separated model P fails, so that P is strictly stronger than the separation.*

The trichotomy places every candidate premise. A premise that cannot force is refuted by a world. A premise that forces and is returned by the separation is the bit under another name. A premise that forces and is not returned is a strict strengthening, and a proof from it proves more than the separation, which is to say it proves the bit and something besides.

10. The Turing model: real machines deciding real SAT, and the classes

Part C of the kernel builds the model. It has five pieces: an encoding of formulas as strings with a decoder proved to invert it, the machines and their step semantics, the running time, the SAT machine of the model together with the proposition SAT $\in$ **P** stated over every Turing machine, and the classes.

The encoding. The input alphabet Σ has three symbols, the binary digits 0 and 1 and a separator #. A variable index n is written as the binary digits of $n+1$, least significant first, which is never the empty string, followed by #; a literal is its index followed by one digit for its sign; a clause is its literals followed by #; a formula is the concatenation of its clauses. The digits are produced by halving and parity defined by two-step recursion in the file, with the recomposition $(\text{digit}) + 2 \cdot \text{half}(n) = n$ proved by the same recursion (`digit_spec`), so that no division operator and no library lemma about division is used anywhere. The decoder reads digits up to a separator (`parseBits`), a literal (`parseLit`), a clause as literals until a separator stands where a literal would start (`parseClause`), and a formula as clauses until the string ends (`parseCNF`), each recursion bounded by a fuel that the length of the string supplies.

Theorem 21 (`decode_encode`, `encode_injective`, no axiom). *For every formula f,* $\text{decode}(\text{encode}\, f) = \text{some}\, f$*; hence distinct formulas have distinct strings.*

The language is then defined on all strings: $\text{satLang}\, w$ is `satB` of the formula w decodes to, and `false` when w decodes to nothing. SAT $\subseteq \Sigma^*$ is the set of strings with $\text{satLang}\, w = \text{true}$, so a malformed string is outside the language, as in the textbook definition.

Theorem 22 (`satLang_encode_iff`, no axiom). *For every formula f,* $\text{satLang}(\text{encode}\, f) = \text{true}$ *if and only if f is satisfiable.*

The machines. A Turing machine is a structure with a number q of states, a number s of work symbols, a total transition function $\delta : \text{Fin}\, q \to \text{Fin}\,(s+4) \to \text{Fin}\, q \times \text{Fin}\,(s+4) \times \{L, R\}$, and three desig-

nated states, start, accept and reject. The tape alphabet Fin $(s+4)$ has the blank at 0, the three input symbols at 1, 2 and 3, and s further work symbols. A configuration is a state, a head position in $\mathbb{Z}$ and a tape, a function from $\mathbb{Z}$ to the tape alphabet; the initial configuration has the input written from cell 0 rightward, blanks elsewhere, the head at 0 and the start state. One step reads the scanned cell, writes, moves one cell and changes state, and a configuration in the accept or reject state is fixed by the step. The run on input w is the iteration of the step from the initial configuration; HaltsAt $M\,w\,t$ says the configuration after t steps is in a halting state, and verdictAt $M\,w\,t$ is `true` exactly when that configuration is in the accept state. The finiteness of the state set is load-bearing and is why the states are a `Fin` type and not a natural number: a machine with infinitely many states could carry the input into its state and decide SAT in linear time, and a model that allowed it would make the separation false for a reason that has nothing to do with computation.

Theorem 23 (`haltsAt_of_le`, `verdict_of_le`, no axiom). *A machine halted at step t is halted at every later step, and its verdict at every later step is its verdict at t.* The proof is an induction on the inequality $t \le u$ itself, which is why no subtraction and no library lemma enters.

The running time. `firstHalt M w n` is the first halting step at or below n, or n when none is found. A **decider** is a machine together with a clock, a function from strings to naturals, and a proof that the machine is halted at the clock's value on every input. The running time of a decider on w is `firstHalt` run up to the clock, and its output is the verdict at that step. The clock certifies and does not time:

Theorem 24 (`time_halts`, `time_least`, no axiom). *The decider is halted at its running time, and its running time is at most every step at which it is halted.* So the running time is the first halting step of the run, whatever clock certified the halting, and the output is the verdict the run itself delivers.

The SAT machine of the Turing model, the classes and the bridge. The machine is

$$\text{TMSAT} := \langle \text{Decider}, \Sigma^*, \text{length}, \text{out}, \text{time}, \text{satLang} \rangle.$$

A language is a function from strings to Booleans. A language L is **in P** when some Turing machine halts on every input within $c\,n^k + c$ steps, n the length of the input, in the accept state exactly on the strings of L:

$$\begin{aligned} \text{InP}\,L := \exists M\,c\,k,\ \forall w,\ \exists t,\\ & t \le \text{bound}(c,k,|w|)\\ & \wedge\ \text{HaltsAt}\,M\,w\,t\\ & \wedge\ \text{verdictAt}\,M\,w\,t = L\,w, \end{aligned}$$

and SATinP $:=$ InP satLang, over every Turing machine, with no decider and no clock in it. A language L is **in NP** when some Turing machine, the verifier, accepts within $c\,n^k + c$ steps the string made of a binary certificate of length at most $c\,n^k + c$, the separator, and the input, exactly when the input is in L; the certificate carries no separator, so the first separator on the tape delimits it:

$$\begin{aligned} \text{InNP}\,L := \exists M\,c\,k,\ \forall w,\ \big(L\,w = \text{true} \leftrightarrow\\ & \exists u : \{0,1\}^*,\ |u| \le \text{bound}(c,k,|w|) \wedge\\ & \exists t \le \text{bound}(c,k,|w|),\ \text{HaltsAt}\,M\,(u \,\#\, w)\,t\\ & \wedge\ \text{verdictAt}\,M\,(u \,\#\, w)\,t = \text{true}\big), \end{aligned}$$

and the sentence is $\mathbf{P} \neq \mathbf{NP} := \exists L,\ \text{InNP}\,L \wedge \neg\,\text{InP}\,L$, the kernel's `P_ne_NP`. These are the textbook definitions on the kernel's model, and all five constants, TMSAT, InP, InNP, $\mathbf{P} \neq \mathbf{NP}$ and SATinP, have empty cones of their own, pinned in Appendix B.

Theorem 25 (`sep_tmsat_iff`, no axiom). Sep TMSAT $\leftrightarrow \neg$ SATinP.

Proof. Left to right: a machine M witnessing SATinP with bound (c,k) is a decider when clocked by that bound, since by Theorem 23 it is halted at the bound; its running time is at most the witnessing step by Theorem 24 and hence within the bound, and its output is the verdict at the witnessing step by Theorems 23 and 24, which is satLang w; so it escapes, against Sep. Right to left: a decider that escapes witnesses SATinP with its own running time as the step. □

Theorem 26 (`evalPoly_le_bound`, `SATinP_of_poly`, no axiom). *Every polynomial with natural coefficients, evaluated by Horner's rule, is at most $S\,n^k + S$ with S its coefficient sum and k its length; hence a machine deciding SAT within any such polynomial witnesses* SATinP. Nothing is lost in the kernel's form of the bound, and the bound is itself a polynomial, so SATinP is polynomial time in the ordinary sense.

Theorem 27 (`pnp_normal_form_turing`, classical). *On the Turing model,*

$$\big(\text{NE}(\text{Rec TMSAT}) \to \text{NE TMSAT}\big) \ \leftrightarrow\ \neg\,\text{SATinP}.$$

This is the normal form of P versus NP on real machines deciding real SAT. The left side is one implication whose antecedent is a theorem (`tmsat_record_has_least_escape`, no axiom). The right side quantifies over every Turing machine and every polynomial bound.

Theorem 28 (`tmsat_record_has_least_escape`, `tmsat_bit_is_the_act`, `tmsat_self_recursion`, `tmsat_coincidence`, no axiom; `tmsat_every_route_passes_the_bit`, `tmsat_posit_is_the_bit`, `tmsat_normal_form_complete`, classical). *Theorems 1, 4, 5, 6, 9, 15, the coincidence form and the identification of Section 12, on the Turing model, with* Sep *replaced throughout by* $\neg$ SATinP.

In particular every route to SAT $\notin$ **P**, from any premise, passes the bit, and the capstone on the Turing model has nothing before the colon at all: it is a closed statement about the model, with SATinP occurring only inside biconditionals and inside the hypothesis of a route.

Theorem 29 (`tmsat_racomp_iff_not_inP`, `the_posit_is_every_exit`, no axiom; `every_exit_is_the_posit`, classical). *The reading of the root on computation, that every decider at every bound fails to escape, is exactly* SAT $\notin$ **P** *on the Turing model. Any proof of* SAT $\notin$ **P**, *by any route, including one built from facts about Turing machines, yields all three names of the one posit: the transfer* NE(Rec TMSAT) $\to$ NE TMSAT, *a least-escape certificate, and* RAcomp TMSAT. *And each of the three names gives* SAT $\notin$ **P** *back, on no axiom.*

So no independent proof exists. Nothing reaches SAT $\notin$ **P** except through least escape on the Turing SAT machine, and least escape gives SAT $\notin$ **P** back; nothing sits between the posit and the conclusion. The one return that is classical is the return from a proof to the transfer, since it passes through the closure; the three returns from the names to the conclusion are axiom-free. The first conjunct is the register's bridge test run inside the kernel: the earlier paper's `ra_comp_is_sep` on the abstract machine becomes, on the Turing model, RAcomp TMSAT $\leftrightarrow \neg$ SATinP with an empty cone.

Theorem 30 (`p_ne_np_of_not_inP`, no axiom). *If* SAT $\in$ **NP** *and* SAT $\notin$ **P**, *then* **P** $\neq$ **NP**.

The theorem is one line, because the classes were defined on one schema, and its content is exactly that: the sentence **P** $\neq$ **NP** of the kernel is reached from SAT $\notin$ **P** by naming one hypothesis, InNP satLang, and nothing else. That hypothesis is the easy half of Cook and Levin's theorem, a verifier machine that evaluates an encoded formula under a certificate assignment in polynomial time. It has been mechanized on other machine models, in Coq [20] and in Isabelle [21], and this paper carries it at citation grade; Section 15 uses it as such.

Theorem 31 (`certificate_unique`, no axiom). *Any two least-escape certificates of the SAT machine are the same object, and any two proofs of* SAT $\notin$ **P** *are the same proof.* By the proof irrelevance of Lean's propositions and the eta rule for structures, every route delivers one object and not a family.

The defense. Three statements are classical and three are not.

Theorem 32 (`tmsat_rejection_is_a_witness`, `tmsat_every_pair_lands`, `forces_or_has_a_twin`, classical). *A rejection of* SAT $\notin$ **P** *is a witness, a decider and a bound with every run clean. Every decider at every bound either is stopped by some string or escapes. Every property of machines either forces the closure or holds on some machine that escapes.*

Theorem 33 (`escape_refutes`, `act_leaves_no_escape`, `extend_escapes`, `no_record_decides`, `tmsat_no_record_decides`, no axiom). *An escape refutes the closure; the closure leaves no escape; and every stock of examinations of a machine with an instance, indexed by any type, finite or infinite, is a stock of stops both on the machine's extension by one algorithm that answers the predicate in no time, where that algorithm escapes, and on the machine's record, where nothing escapes.*

Theorem 33 is the generalization of Stage 1's locality from one observation to all of them. Whatever stops have been observed, there is a machine agreeing with every one of them on which SAT is easy and a machine agreeing with every one of them on which nothing escapes. Observed stops decide nothing; the universal is never a sum. The machine consistent with every stock is `extendM TMSAT`, the SAT machine with one algorithm added, and its record is `Rec TMSAT`.

What the model does and does not fix. The machine is single-tape, two-way infinite, with a three-symbol input alphabet; the textbook classes are stated for one-way tapes, several tapes, or a binary input alphabet, and polynomial time is the same under every one of these variants by the standard simulations, which cost a polynomial factor and are proved in Sipser [13, Theorem 7.8] and Arora and Barak [11, Section 1.3]; the paper cites them and does not formalize them. The encoding of formulas is one of the usual ones, with indices in binary, and polynomial time is the same under every encoding computable in polynomial time from this one and back, as all the usual encodings are. The classes are defined on the kernel's model; their identification with the classes of the textbooks is the robustness just cited. These citations are the whole distance between Theorem 30 and the sentence **P** $\neq$ **NP** as a complexity theorist writes it, and each is a settled theorem of the field rather than a premise of this paper.

Executable checks. Fourteen examples, closed by `decide` inside the compile, run the model: a three-state machine that accepts exactly when the first input symbol is 1 is not halted at step 0, halts after one step with the right verdict on `[1]` and on `[0]`, halts on the empty input, and keeps its verdict at step 5 on `[1,0]`; the encoding of the one-literal formula x_0 is the string 1#1#, which decodes back to it; the string 11 decodes to nothing; the empty string is in SAT (the empty formula), the string # is not (one empty clause), the formula x_0 is, the formula $x_0 \wedge \neg x_0$ is not, and $(x_1 \vee \neg x_0) \wedge \neg x_1$ is. Altering the expected value of any of them stops the compile, and Appendix B records the compiler's refusal for one of them.

11. The frontier, formal alone

Part D carries, from the register's earlier kernel on this row, the two frontier results that are pure mathematics and need no physics and no root: an axiom-free pigeonhole bound on black-box search, and the white-box contrast that shows what the bound does not reach.

Theorem 34 (`pigeon`, `black_box_bound`, `black_box_no_savings`, no axiom). *A list of naturals that contains every number below K has at least K entries. A fixed list of tests that decides, for every candidate function $f : \mathbb{N} \to \{\text{true}, \text{false}\}$, whether some candidate below K satisfies f, by answering `true` exactly when some tested candidate does, has length at least K; over 2^N candidates it is never shorter than 2^N.*

The theorem is exact about its scope: the procedure is a fixed list of queries, and its verdict is the disjunction of the answers. That is the test-only procedure on the all-false path of the standard adversary argument, where adaptivity collapses because every answer is `false`; the kernel formalizes the fixed list and the paper does not claim the adaptive statement beyond it. On 2^n truth assignments the bound is exponential and it is a theorem on no axiom.

Theorem 35 (`reader_solves`, `reader_poly`, `fails_when_descriptions_reveal`, `missing_object_is_keyed`, no axiom). *The black-box adversary family is the family of point functions. Handed over as descriptions, each is solved by a reader of its description in one step, in constant time. Hence the white-box object, that every algorithm deciding the problem from its description runs in super-polynomial time, fails on the model where descriptions reveal, and holds on every model where no algorithm decides: it is keyed.*

The two theorems together say where the frontier is. Black-box bounds are exponential and unconditional and say nothing about SAT $\notin$ **P**, because the Turing machine reads the formula; the object that would say something is a lower bound against readers of the description, and that object is keyed exactly as the bit is. This is the formal-alone content of the frontier the register's earlier paper carried with physics beside it; here the physics enters once, in Section 15, as the carrier premise of the witness, and nowhere in the form.

12. The residue

Part E of the kernel prints what remains.

Theorem 36 (`separation_of_the_act`, no axiom). *For every machine M with an instance, given the transfer* NothingEscapes(Rec M) $\to$ NothingEscapes M, *the separation* Sep M *follows.*

Theorem 37 (`sat_separation_of_the_act`, no axiom). *In every algorithm model, given the transfer on the SAT machine, SAT is separated in that model.*

Theorem 38 (`sat_not_in_P_of_the_act`, no axiom). *Given the transfer on the SAT machine of the Turing model,* $\neg$ SATinP: *no Turing machine decides SAT within any polynomial bound.*

Theorem 39 (`p_ne_np_of_residue`, no axiom). *Given* SAT $\in$ **NP** *and the transfer on the SAT machine of the Turing model,* **P** $\neq$ **NP**.

The hypothesis of Theorem 38 is the residue of P versus NP in this paper. It is not assumed anywhere in the file, as the reader can confirm by searching Appendix A for `axiom`, and it is the only thing standing between the theorems above and SAT $\notin$ **P**; with SAT $\in$ **NP** cited beside it, it is the only thing standing between them and the sentence **P** $\neq$ **NP**. Sections 4 to 14 do not posit it and do not state its value; Section 15 supplies it, at premise grade, and states its value.

Theorem 40 (`residue_is_keyed`, no axiom). *No keyless property forces the closure of every machine; no principle true of every record decides every machine; no property shared by every algorithm model of SAT forces the closure of any.*

Theorem 40 restates the wall on the residue. The hypothesis of Theorem 38 cannot be supplied by anything shared by all machines, by anything true of every record, or by anything shared by all algorithm models of satisfiability itself. What supplies it, if anything does, is a fact about the Turing model that fails in a world where one algorithm answers satisfiability in no time. The register this paper is drawn from [14] reads that fact as an act and identifies the reading of its root principle on computation with the transfer; the kernel carries that identification as `the_posit_is_the_bit` and `tmsat_posit_is_the_bit`, which prove that the statement *every machine at every bound fails to escape*, the register's own form of the principle, is exactly the transfer. The reading supplies no theorem in Sections 4 to 14, whose claims do not depend on it; it is the posit of Section 15. In the paper's own terms the residue is a type, printed, and supplied in Section 15, and the guard sentence of the register reads: everything except the bit is proved with no posit, the constructive parts on no axiom at all and the classical return from the separation to the closure on Lean's standard axioms alone; and the bit, the transfer of least escape from the record of the SAT machine to the SAT machine, equivalent to SAT $\notin$ **P** over every Turing machine and, with SAT $\in$ **NP**, to **P** $\neq$ **NP**, is all that remains, at premise grade.

The two defenses of the row follow from Theorems 25 and 27 and are stated once. Nothing passes for the separation but the transfer, since the normal form is an equivalence. Nothing passes against it but an escape, a Turing machine, a bound and every run clean, since by Theorem 32 the rejection of $\neg$ SATinP is exactly that existential. The one refuter of the residue is therefore the familiar one: a polynomial-time Turing machine for satisfiability. The kernel's theorems are refuted by nothing of the kind, because they hold in the world where such a machine exists as well as in the world

where it does not.

13. Falsification criteria

Three criteria, each forced by the paper's own construction, each aimed at a claim the paper makes and at nothing it does not.

Criterion 1, the silent build. The paper's claims are the pinned cones of Appendix B. Recompile Appendix A under Lean 4.19.0 with warnings fatal (`lean -DwarningAsError=true PNP_Normal_Form_Paper_v1_3_0_CC.lean`). The forcing step: every theorem numbered above is a theorem of that file; each of the 93 pinned constants carries the cone the paper states as a docstring that the compiler compares with its own output; the fourteen executable checks are closed by the compiler or the compile fails; and a planted `sorry` is a warning, which the flag makes an error. Confirmation: exit code 0 and no output. Null hypothesis: any output at all, a nonzero exit, a `#guard_msgs` mismatch on any pin, a failed check, or a warning, refutes the paper as a whole, since the paper claims nothing beyond the pinned lines. Blast radius: the paper. A failure here cannot reach the companion master or the register, which carry their own receipts. Census of the pins: 72 read *does not depend on any axioms* and 21 read `[propext, Classical.choice, Quot.sound]`; no other cone occurs.

Criterion 2, the finite battery. Theorems 1, 2, 4, 6, 8, 13, 14 and 19 have finite content: on any finite machine, with a finite grid of bounds, each is a decidable equation between computed Booleans, and Theorems 10 and 19 are witnessed on any zoo that contains a stage world. The forcing step: the theorems are equivalences and refutations that quantify over the machine, so any finite machine is a test instance, and the Turing model of Section 10 adds its own finite instances, the fourteen checks, which any reader can extend by writing a machine and a string. Confirmation: on every finite machine and every finite grid, the transfer computed as a Boolean equals the separation computed as a Boolean, the record's closure computes `true`, the closed and stage worlds give the two values, and the planted inflations, the transfer asserted on the stage world, a fixed point of negation, the record's self-recursion carried to the machine, each compute `false`; on the model, every run of a written machine on a written string gives the verdict and the halting step the step function dictates, and every encoded formula decodes to itself. The author's witness reports the abstract part on 45 machines at the seed 20260622 with 12 checks passed and 3 planted failures; the model part is the fourteen checks in the compile. Null hypothesis: a finite machine with an instance on which the Boolean transfer and the Boolean separation differ refutes Theorem 2, and a written machine and string on which the run disagrees with the step function, or a formula that does not decode to itself, refutes Part C; since each is a theorem of the kernel or a computation it performs, such a case would witness a defect in Lean's kernel, which is the honest statement of what this criterion tests. Blast radius: the kernel and the Lean checker. The criterion exists because a reader without Lean can run the abstract part in any language in an afternoon.

Criterion 3, the refuter of the value. The forcing step: Section 15 states the value at premise grade, delivered by two kinds that Theorem 45 proves to be one bit. Confirmation: no Turing machine, polynomial bound and set of runs all clean is known. Null hypothesis: one escape, a Turing machine, a bound and every run clean, refutes both kinds at once (Theorem 47) and with them the value; it leaves every theorem of the kernel standing, since each is an equivalence, a wall or a conditional. Blast radius: the value of Section 15, never the form.

14. Discussion

The normal form changes what a proof of $\mathbf{P} \neq \mathbf{NP}$ is required to produce. It is required to produce one implication on one concrete machine, TMSAT, and Theorems 38 and 39 show that producing it is enough. Every known or future route is a route to that implication, by Theorem 28, so the field's accumulated analysis of routes, which barrier each evades and which premise each needs, can be read as an analysis of ways to deliver the bit. The paper proposes no route. It fixes the target.

The keyless wall is weaker than the three barriers and should not be mistaken for a fourth. What it has that they do not is unconditionality at zero cost: it is a theorem with an empty cone, in a file with no axioms, about every machine and now about every algorithm model of SAT. The barriers say that specific powerful techniques fail; the wall says that the reason any technique must be keyed to the actual model is already visible at the level of the definitions. A proof that does not use some fact distinguishing the Turing model from the stage SAT world does not prove the separation. One consequence is practical: a candidate proof can be screened by asking which of its premises fails in a world where a single algorithm decides SAT in zero steps. If none does, the proof is uniform in the model and Theorem 19 refutes it before any detail is read. The trichotomy of Theorem 20 is the same screen with its outcomes named.

The model construction is the part of the paper a complexity theorist will check first, and it is built to be checked. The machines are the textbook objects with their finiteness enforced by the type, the semantics is a step function with the halting states absorbing, the running time is the first halting step and is proved independent of the certificate that supplied it,

the encoding is inverted by a decoder, SAT is a language on all strings, the classes are the textbook definitions, and SATinP quantifies over every machine. The one design choice that is not textbook is the decider's clock, and Theorem 24 shows it is inert: it certifies that the run halts and has no other effect. Theorem 25 then removes the deciders from the final statement altogether.

The axiom discipline is the second thing such a reader will check. The file was written so that the only classical results are the twenty-one that pass from the separation back to the closure, assert a disjunction or a double negation over an infinite type, or state the tautology of Theorem 7, and everything else, the decision procedure, the encoding and its decoder, the machines, the running time, the bridge, the classes, the frontier and the residue, is proved with an empty cone. That required proving in the file the few arithmetic and list facts that the core library proves with classical tactics, by structural induction; the cost was a hundred lines and the gain is that the headline theorems of the paper, `sep_tmsat_iff`, `decode_encode`, `sat_bit_is_keyed`, `sat_keyless_forces_nothing`, `sat_not_in_P_of_the_act` and `p_ne_np_of_residue`, depend on nothing but the Lean kernel's type theory.

The comparison with the Riemann normal form is the paper's one claim of structural generality, and it is stated at theorem grade and no higher. In the companion master [15] the hypothesis of the line is equivalent to the transfer of least erasure from the record of the zero set to the zero set, the record has the property unconditionally, the transfer is the act, the record is self-recursive and its self-recursion does not transfer, every route passes the bit, and the record decides nothing. The present kernel proves the counterpart of each of these on the machine, and now on the Turing machine. The two rows differ at exactly one point, the seat: the fold of the critical strip has a fixed line and the involution of the two computational worlds has no fixed point, which is why one question admits a finite verification to a height and the other admits no finite verification at all. Theorem 14 forbids reading the shared shape as a shared bit.

Three limitations are stated plainly. First, the kernel formalizes one machine variant and one encoding, and carries SAT $\in$ **NP** as a cited theorem of Cook and Levin, mechanized on other models in Coq and Isabelle [20, 21]; the robustness of polynomial time under the other variants and encodings is cited likewise. Second, the logic of the normal form is elementary; the paper's contribution is the formal fixing of the question's structure on a concrete model, with the classes defined, not a new theorem about algorithms. Third, the value of the separation is supplied in Section 15 at premise grade, on the one posit; the form, Sections 4 to 12, does not depend on it, and a polynomial-time Turing machine for SAT would refute the value and leave the form standing.

An objection should be met directly. One might say that a normal form which reduces to the classical equivalence between $\neg\exists$ and $\forall\neg$ has no content. The reply is that the content is in what is fixed, not in what is derived: the paper fixes, formally, with no premise and on Turing machines deciding SAT, that the separation is a single transfer with a proved antecedent, that the transfer is keyed on the algorithms alone, that nothing keyless, nothing record-level and nothing uniform over the algorithm models of SAT supplies it, that every route delivers exactly it and delivers one object, and that every stock of observations is consistent with both values. Each of these is a claim one finds in the literature in informal form, and none had a pinned cone before. The paper's work is to make them theorems, to attach the file, and to say exactly how strong they are.

How the paper stands to each prior position. The paper sits across the founding theorems of the question, the three barriers, the one open algorithmic route, the hierarchy, proof complexity, geometric complexity, black-box search, the model-theoretic organization of the possible worlds, the history, the textbooks that fix the model, the proof assistant, and its own two companions, and it owes each a statement of relation in a fixed vocabulary. *Additive* means the paper supplies a result or a test the position lacked and leaves it standing. *Replacing* means a framing is retired and a measured object put in its place. *Subsuming* means the prior claim becomes a case of the paper's object. *Corroborating* means independent agreement, which confers no warrant on either side. *Contradicting* means a named thesis is denied on executed data, never on argument. *Competing* means a different position on the same question, argued. *Scoping* means the prior claim is kept inside a stated boundary. *Kin* means a position the paper is continuous with. *Superseding* is permitted only where a named theory's central claim is retired on executed data at the same register; it is used nowhere in Table 1, and the paper supersedes no theory. The census of Table 1: the paper scopes five positions, Cook and Levin, Williams, Fortnow, Hartmanis and Stearns, and the textbooks of Sipser and of Arora and Barak; is kin to nine, Karp, Baker, Gill and Solovay, Razborov and Rudich, Aaronson and Wigderson, Impagliazzo, Gödel through Sipser, Cook and Reckhow, the mechanizations of Gäher and Kunze, Balbach and lean-dojo, and its companion master, the last with the relation executed as Theorem 14; competes with one, Mulmuley and Sohoni, without executing a comparison; and adds to three, deterministic query search, Lean 4, and the register's earlier paper on this row. The paper's contribution is exactly the set of relations the table states and nothing wider.

Table 1 | How the paper stands to each prior position. Relation words as defined in the text; evidence in one word, never above the tier the body carries.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973	SAT is NP-complete; P versus NP is the question of SAT in P.	Proves the normal form of SAT $\in$ P over all Turing machines, defines the classes, and reaches P $\neq$ NP from SAT $\notin$ P with SAT $\in$ NP cited from them.	scoping	executed
Karp 1972	NP-completeness is generic across combinatorial problems.	Explains why one language carries the question; no reduction is formalized.	kin	cited
Baker, Gill, Solovay 1975	Relativizing proofs cannot settle P versus NP.	Proves a schema-level relative, the keyless wall, now on the algorithm models of SAT, with an empty cone; claims no barrier.	kin	executed
Razborov, Rudich 1997	Natural proofs cannot prove strong circuit lower bounds, given pseudorandomness.	Cited as a barrier on a technique; the paper proves nothing about circuits.	kin	cited
Aaronson, Wigderson 2009	Algebrizing techniques cannot settle P versus NP.	Cited as a barrier on a technique; the paper's wall is independent of oracles and extensions.	kin	cited
Williams 2014	A faster circuit-SAT algorithm yields a circuit lower bound; NEXP is not in ACC^0.	Keeps the route open and shows, by Theorem 28, that it passes the bit like every route.	scoping	cited
Hartmanis, Stearns 1965	More time decides more languages.	Keeps the hierarchy's separations to resource bounds of different order, reached by arguments that relativize.	scoping	cited
Cook, Reckhow 1979	NP $\neq$ coNP exactly when no proof system is polynomially bounded.	Stands beside it as a second conversion of a separation into one object, here proved equivalent by machine.	kin	cited
Mulmuley, Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	A different route to the same bit, argued, without an executed comparison.	competing	cited
Deterministic query search	Black-box search over N candidates needs N tests on the all-false path.	Compiles the bound for fixed test lists by pigeonhole on no axiom, and the white-box reader that beats it in one step.	additive	executed
Impagliazzo 1995	Five possible worlds organize the question.	Uses coherent worlds of a different kind, now on the formulas themselves, to prove the bit is keyed.	kin	executed
Gödel 1956, in Sipser 1992	The question is the cost of finding against the cost of checking.	Carries the asymmetry as the examination structure of Stage 1.	kin	documentary
Fortnow 2009	The question is open and the barriers stand.	Holds the status unchanged and adds the formal residue.	scoping	cited
Sipser 2013; Arora, Barak 2009	The Turing machine model, and polynomial time robust under its variants and encodings.	Builds one variant and one encoding in full, defines the classes on it; cites the robustness and does not reprove it.	scoping	executed
de Moura, Ullrich 2021	Lean 4 checks proofs against a small kernel.	Adds a core-Lean kernel for the row, with a Turing model inside it, no imported library, and every cone pinned.	additive	executed
Gäher, Kunze 2021; Balbach 2023; lean-dojo	Cook and Levin mechanized in Coq and Isabelle; P versus NP stated in Lean 4 on a Turing model.	Kin to all three; cites the mechanized SAT $\in$ NP and adds the normal form, the wall and the supply on a core-Lean model with no import.	kin	cited
Islam 2026, closure on one posit	The separation closed on an abstract machine, the value on a posit.	Carries its pins, defense, formal-alone frontier and the one posit onto the real machine, the posit supplied in Section 15; leaves its physics frames and fielded routes in the register.	additive	executed
Islam 2026, formal master	The Riemann hypothesis is one transfer from a record with least erasure.	Proves the counterpart of each statement on the machine and separates the two bits by Theorem 14.	kin	executed

15. The supply: the bit delivered, witnessed, confirmed and closed

Sections 4 to 14 prove everything except the bit, with no posit. This section supplies the bit and closes the row. It is the only place in the paper where a premise enters, and every theorem that uses the premise prints it as a hypothesis. Part F of the kernel carries the section: eight theorems compiled silently in the same file,

six on no axiom and two on Lean's standard three, every cone pinned.

Supplied. The bit is one implication: the transfer of least escape from the record of the Turing SAT machine to the machine itself. By `tmsat_posit_is_the_bit` and `tmsat_racomp_iff_not_inP` it is exactly the register's root read on computation: every algorithm at every bound is stopped, which is to say that every decider of satisfiability actuates beyond every polynomial bound. The register [14] holds that reading as its one posit. Here it is supplied by the act, at premise grade, and derived from nothing. The wall of Theorem 40 proves that nothing keyless, nothing record-level and nothing uniform over the algorithm models of SAT can derive it, so a supply is the only thing that can carry it.

Delivered, twice. The bit arrives by two kinds. Each is a type, and an inhabitant of that type is the supply.

Kind 1, the formal cargo. The record's half arrives with no premise.

Theorem 41 (`cargo_arrives`, no axiom). *The record of the Turing SAT machine has least escape.*

The cargo is the transfer itself, `FormalCargo`, and it delivers.

Theorem 42 (`formal_kind_delivers`, no axiom). *The formal cargo gives* SAT $\notin$ **P** *in one step.*

Kind 2, the physical witness. Every run of a decider is a physical process. Each step registers a record, and each registration costs at least $k_B T \ln 2$ (Landauer [22]; measured by Bérut and colleagues [23]). The witness is a structure with four parts: the floor, positive; the heat of each run, counted in floor units; that heat priced at the run's step count; and one field that reads the bit in the register of heat, that no decider answers every formula correctly within a polynomial heat budget. The last field is the posit stated in physical vocabulary, at premise grade, and it is a structure field, assumed nowhere in the file.

Theorem 43 (`witness_closes`, no axiom). *A physical witness stops every decider at every bound:* NothingEscapes TMSAT.

Theorem 44 (`physical_kind_delivers`, no axiom). *A physical witness gives* SAT $\notin$ **P**.

The measured floor is corroboration under its layer tag, consensus, and is never warrant; the kernel never uses its value. What the witness contributes is the reading of the bit in a second register, heat, and the proof that the reading closes the machine.

Witnessed. The two kinds are one bit.

Theorem 45 (`the_two_kinds_are_one_bit`, classical). *Every physical witness is a formal cargo, and the cargo is exactly* SAT $\notin$ **P**.

By `every_exit_is_the_posit` and `certificate_unique` there is no third kind and no second certificate: whatever delivers the separation, by any route, delivers this one bit and is this one object. Beside the witness stands the verify-find asymmetry, which the kernel carries as `black_box_bound`: finding is dear and checking is cheap. It is corroboration. Evidence leans; it does not rule.

Confirmed. With SAT $\in$ **NP**, either kind gives the sentence.

Theorem 46 (`p_ne_np_delivered_twice`, no axiom). *Given* SAT $\in$ **NP**, *the formal cargo gives* **P** $\neq$ **NP**, *and so does a physical witness.*

SAT $\in$ **NP** is the easy half of the theorem of Cook [1] and Levin [2]; it has been mechanized on other machine models, by Gäher and Kunze in Coq [20] and by Balbach in Isabelle [21], and this paper carries it at citation grade. The double defense confirms the supply from the other side. Nothing passes against it except an escape (Theorem 32), and one escape refutes both kinds at once.

Theorem 47 (`the_one_refuter`, no axiom). *An escape, a decider and a bound with every run clean, refutes the formal cargo and the physical witness together.*

No such machine is known.

Closed. One statement binds the section, with nothing before the colon.

Theorem 48 (`the_supply_closed`, classical). *The cargo arrives free; each kind delivers* SAT $\notin$ **P**; *every physical witness is a formal cargo; the cargo is exactly* SAT $\notin$ **P**; *given* SAT $\in$ **NP**, *each kind gives* **P** $\neq$ **NP**; *and one escape refutes both kinds.*

The face of the row is therefore this. The form, at theorem grade, on real Turing machines deciding real SAT. The value, **P** $\neq$ **NP**, at premise grade, supplied by the act on the one posit and delivered twice, by the formal cargo and by the physical witness. The refuter, one polynomial-time Turing machine for satisfiability. The row is closed, and the register marks it with the dot.

16. Conclusion

The gap was a missing form. The question of P versus NP had barriers against techniques and no formal statement of what any technique must produce. The kernel attached to this paper produces it, and produces it on the real object: Turing machines with finite control and a tape, deciding the language of encoded satisfiable formulas, with the classes defined. The separation, stated over every Turing machine as ¬ SATinP, is one implication, the transfer of least escape from the record of the SAT machine to the SAT machine, the record's half proved with no axiom, and the implication keyed on the algorithms alone, unreachable from anything shared by all machines or by all algorithm models of SAT, delivered by every route and only by a route as one object, consistent with every stock of observations, and seated on no fixed point. With SAT $\in$ **NP**, cited, the implication gives the sen-

tence $\mathbf{P} \neq \mathbf{NP}$, and Section 15 supplies the implication by the act, delivered by the formal cargo and witnessed by the physical carrier, at premise grade; the row closes on the dot.

The primary falsifier is the silent build. Recompile the attached file under Lean 4.19.0 with warnings fatal; the paper stands or falls on exit code 0 and no output.

The community is asked for two executions. The first is the recompilation, which any reader can perform in seconds. The second is the verifier: a Turing machine of the kernel's model that evaluates an encoded formula under a certificate assignment, with its correctness and its polynomial step bound proved, which carries $\mathrm{SAT} \in \mathbf{NP}$ into the kernel's own model in place of the citation; the full Cook and Levin reduction, and the robustness of polynomial time under the variants of the model, are the executions after that. The open question that remains is the only one there ever was, now stated as a type: whether any fact about the Turing model that fails where one algorithm answers satisfiability in no time forces the transfer, which is to ask whether anything short of the separation implies it.

If the normal form is accepted, the reframing is this: the question is no longer which technique can prove the separation but which keyed fact delivers one implication on one machine, and every technique is to be read as a candidate carrier of that fact and nothing else.

A. The kernel, verbatim

The file `PNP_Normal_Form_Paper_v1_3_0_CC.lean`, 1,691 lines, SHA-256 `97af802524b5134e 026a4dbff7b1c127 3f4c3e263e5dcdc7 d413a7a2998b0a6c`, as compiled, set in two columns with its line numbers; a line longer than the column continues on the next line without a number. Part A is the kernel `PNP_Normal_Form_CC.lean` of 21 October 2026 carried verbatim; Part B is the SAT instance in every algorithm model with the zero-axiom helpers and the SAT-fixed worlds; Part C is the Turing model with the classes, the coincidence form, the unique certificate and the defense; Part D is the formal-alone frontier; Part E is the residue; Part F is the supply; the pins close each part.

```
/-
  PNP_Normal_Form_Paper_v1_3_0_CC.lean · the kernel of the paper
  "P ≠ NP Closed to One Bit: The Unconditional Normal Form of the
  Separation of SAT on Turing Machines".
  Claude-coded (_CC). Core Lean 4.19.0: no import, no axiom declared,
  no sorry, no native_decide, no posit,
  no physics and no root in Parts A to E: formal alone to the bit;
  Part F supplies it.

  Part A (namespace PNP.NORMALFORM) is PNP_Normal_Form_CC.lean of 21
  October 2026, carried verbatim
  (sha256 33c8d9dd601b9729a650b4bcd67e68ddd81740325d1c2fdfa0aad27049457
  933): the machine, the closure, the act,
  the record, the normal form, the cargo loop, self-recursion, the two
  coherent worlds, the keyless wall, Stage 1,
  the vacant seat, two rows and two bits.
  Part B (namespaces PNP.ZA, PNP.SAT) makes the predicate concrete on
  no axiom: zero-axiom list and arithmetic
  helpers; formulas in conjunctive normal form, satisfiability by
  exhaustion proved sound and complete; the SAT
  machine over any algorithm model; the SAT-fixed worlds and the
  keyless wall on the formulas themselves; the
  premise trichotomy.
  Part C (namespace PNP.TM) makes the model concrete on no axiom: the
  binary encoding and its decoder; Turing
  machines with finite control, one two-way tape and a total
  transition function; the running time as the first
  halting step; SAT as a language; the SAT machine of the Turing
  model; the classes P and NP as definitions and
  the sentence P ≠ NP; the bridge, over every Turing machine, that the
  separation is exactly SAT ∉ P; the normal
  form on the model; the coincidence form; the unique certificate; the
  defense; executable checks.
  Part D (namespaces PNP.BLACKBOX, PNP.MISSING) carries the
  formal-alone frontier of the register: the
  black-box floor by pigeonhole and the white-box contrast.
  Part E (namespace PNP.RESIDUE) prints the one implication that
  remains as a hypothesis in a theorem, and the
  sentence P ≠ NP as its consequence under SAT ∈ NP, so that the
  residue is a type and is assumed nowhere.
  Part F (namespace PNP.SUPPLY) supplies the bit by its two kinds, the
  formal cargo and the physical witness, each
  a type whose inhabitant is the supply, and closes the row; the
  supply is a hypothesis of every theorem that uses it.
  Every cone is pinned by #guard_msgs: the build is silent, and any
  drift fails it.
  No seat is earned by this file. ΔM = 0.
-/

/-! # PART A · THE NORMAL FORM, carried verbatim -/
namespace PNP.NORMALFORM

/-! ## 0 · the machine, the closure, the act, verbatim from
    PNP.HARDENED -/
structure Machine where
  Alg  : Type
  Inst : Type
  size : Inst → Nat
  out  : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat  : Inst → Bool

def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
  M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean
    M A c k x
def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c
    k x
def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k

structure LeastEscape (M : Machine) where
  closure : NothingEscapes M

structure Examination (M : Machine) where
  A : M.Alg
  c : Nat
  k : Nat
  x : M.Inst
  stops : ¬ Clean M A c k x

theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
  fun (A, c, k, he) => match h A c k with
    | (x, hx) => hx (he x)

theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
  fun A c k => Classical.byContradiction fun hn =>
    h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x,
    hx))

/-! ## I · the record: the registration of every pair onto its stop -/

theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide

/-- THE RECORD OF A MACHINE. The registration keeps the algorithms,
    the instances, the sizes and the times and
    forgets the side of every pair: on the record every output is read
    against satisfiability as its opposite, so
    no check is clean. The record is the machine read onto its stops,
    as the record of a zero set is the zero set
    read onto the line. -/
def Rec (M : Machine) : Machine :=
  { M with out := fun _ x => !(M.sat x) }

/-- THE RECORD HAS LEAST ESCAPE, UNCONDITIONALLY: on the record no
    check is clean, so every pair stops at every
    instance (the mirror of `record_has_least_erasure`). One instance
    is needed to stop at. -/
theorem record_has_least_escape (M : Machine) (x₀ : M.Inst) :
    NothingEscapes (Rec M) :=
  fun _ _ _ => (x₀, fun hc => bnot_ne (M.sat x₀) hc.1)

/-- The record is separated, with no premise but an instance (the
    mirror of `cargo_at_zeros`). -/
theorem record_is_separated (M : Machine) (x₀ : M.Inst) : Sep (Rec M)
    :=
  sep_of_closure (Rec M) (record_has_least_escape M x₀)

/-! ## II · the normal form: the separation is one implication -/

/-- THE NORMAL FORM OF P VERSUS NP (the mirror of `zeta_ra_loop`,
    first leg): the separation is exactly the
    transfer of least escape from the record of the machine to the
    machine. The antecedent is a theorem, so the
    implication carries the whole content of the separation: it is the
    1-bit, and nothing else remains. -/
theorem sep_normal_form (M : Machine) (x₀ : M.Inst) :
    (NothingEscapes (Rec M) → NothingEscapes M) ↔ Sep M :=
  (fun h => sep_of_closure M (h (record_has_least_escape M x₀)),
   fun hs _ => closure_of_sep M hs)

/-- THE CARGO LOOP: the record's least escape, unconditional, carried
    by the one bit, is the separation
    (the mirror of `rh_from_the_cargo_loop`; the bit is the printed
    premise). -/
theorem sep_from_the_cargo_loop (M : Machine) (x₀ : M.Inst)
    (bit : NothingEscapes (Rec M) → NothingEscapes M) : Sep M :=
  sep_of_closure M (bit (record_has_least_escape M x₀))

/-- The bit is the act: the transfer holds exactly when the act
    stands. -/
theorem bit_is_the_act (M : Machine) (x₀ : M.Inst) :
    (NothingEscapes (Rec M) → NothingEscapes M) ↔ Nonempty
    (LeastEscape M) :=
  (fun h => ⟨(h (record_has_least_escape M x₀)), fun ⟨L⟩ _ =>
    L.closure⟩)

/-! ## III · self-recursion: a machine that coincides with its own
    record -/

/-- The profile of a machine: which pairs escape. The record's profile
    is empty. -/
def SelfRecursive (M : Machine) : Prop := ∀ A c k, Escapes M A c k ↔
    False

/-- THE RECORD IS SELF-RECURSIVE, with no premise but an instance (the
    mirror of `record_is_self_recursive`). -/
theorem record_is_self_recursive (M : Machine) (x₀ : M.Inst) :
    SelfRecursive (Rec M) :=
  fun _ _ _ => (fun he => bnot_ne (M.sat x₀) (he x₀).1, fun h =>
    h.elim)

/-- THE MACHINE IS SELF-RECURSIVE EXACTLY WHEN IT IS SEPARATED (the
```

```
    mirror of `self_recursion_at_zeros`). -/
theorem self_recursion_at_machine (M : Machine) : SelfRecursive M ↔
    Sep M :=
  (fun h ⟨A, c, k, he⟩ => (h A c k).mp he, fun hs A c k => ⟨fun he =>
    hs ⟨A, c, k, he⟩, fun h => h.elim⟩)

/-- A statement self-verifies exactly when it is true (the mirror of
    `self_verifying_iff_true`). -/
theorem self_verifying_iff_true (P : Prop) : (¬ P → P) ↔ P :=
  (fun h => Classical.byCases id h, fun hp _ => hp)

/-! ## IV · the two coherent machines: the bit is keyed, the record's
    self-recursion does not transfer -/

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
  | 0 => rfl
  | n + 1 => beq_self n

def stageWorld (n : Nat) : Machine :=
  (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ _ => 0, fun _
    => true)
def closedWorld : Machine :=
  (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ =>
    true)

theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
  fun _ => ⟨beq_self n, Nat.zero_le _⟩
theorem closed_world_closes : NothingEscapes closedWorld :=
  fun _ _ _ => ⟨(), fun ⟨h, _⟩ => Bool.noConfusion h⟩
theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld
    n) :=
  fun h => sep_of_closure _ h ⟨n, 0, 0, stage_escapes n⟩

/-- THE BIT IS KEYED: the transfer holds on the closed world and fails
    on the stage world. -/
theorem bit_is_keyed :
    (NothingEscapes (Rec closedWorld) → NothingEscapes closedWorld) ∧
    ¬ (NothingEscapes (Rec (stageWorld 0)) → NothingEscapes
    (stageWorld 0)) :=
  ⟨fun _ => closed_world_closes,
   fun h => stage_fails_closure 0 (h (record_has_least_escape
    (stageWorld 0) ()))⟩

/-- THE RECORD'S SELF-RECURSION DOES NOT TRANSFER (the mirror of
    `record_self_recursion_does_not_transfer`):
    the record of the stage world is self-recursive and the stage
    world is not. -/
theorem record_self_recursion_does_not_transfer :
    SelfRecursive (Rec (stageWorld 0)) ∧ ¬ SelfRecursive (stageWorld
    0) :=
  ⟨record_is_self_recursive (stageWorld 0) (),
   fun h => (h (0 : Nat) 0 0).mp (stage_escapes 0)⟩

/-- CLOSURE OF THE RECORD GIVES NO SELF-RECURSION OF THE MACHINE: no
    reading of the record decides the machine
    (the mirror of `closure_does_not_give_self_recursion`,
    `record_decides_nothing`). -/
theorem record_decides_nothing :
    NothingEscapes (Rec closedWorld) ∧ NothingEscapes (Rec (stageWorld
    0)) ∧
    NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0) :=
  ⟨record_has_least_escape closedWorld (), record_has_least_escape
    (stageWorld 0) (),
   closed_world_closes, stage_fails_closure 0⟩

/-! ## V · every route passes the bit; keyless premises force nothing
    -/

/-- EVERY ROUTE PASSES THE BIT (the mirror of
    `every_route_passes_the_bit`): any premise that yields the
    separation yields the transfer, so every proof of P ≠ NP, from any
    premise, delivers the one implication. -/
theorem every_route_passes_the_bit (M : Machine) (x₀ : M.Inst) (Q :
    Prop) (h : Q → Sep M) :
    Q → (NothingEscapes (Rec M) → NothingEscapes M) :=
  fun q => (sep_normal_form M x₀).mpr (h q)

def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M

/-- A KEYLESS PREMISE FORCES NOTHING (the mirror of
    `the_unconditional_route_is_blocked`): what every machine
    shares yields the transfer on none. -/
theorem keyless_forces_nothing (P : Machine → Prop) (hP : Keyless P) :
    ¬ Forces P :=
  fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))

/-- NO CLOSED PRINCIPLE DECIDES: a principle true of the record of
    every machine decides the machine on none
    (the mirror of `no_closed_principle_decides`). -/
theorem no_record_principle_decides (P : Machine → Prop) (hP : ∀ M, P
    (Rec M)) :
    ¬ ∀ M, P (Rec M) → NothingEscapes M :=
  fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))

/-! ## VI · the cargo: every examination enacts Stage 1,
    unconditionally -/

/-- STAGE 1 OF THE ROW: a pair examined and stopped. -/
structure Stage1 (M : Machine) where
  e : Examination M
  stopped : ¬ Clean M e.A e.c e.k e.x
  witnessed : ∃ x, ¬ Clean M e.A e.c e.k x

/-- THE EXAMINATION ENACTS STAGE 1: every stopping run supplies one
    (the mirror of
    `computation_enacts_stage1`); no universal appears in it. -/
def examination_enacts_stage1 (M : Machine) (e : Examination M) :
    Stage1 M :=
  ⟨e, e.stops, (e.x, e.stops)⟩

/-- Stage 1 is local: it decides its pair and no other (the universal
    is the act). -/
theorem stage1_is_local (M : Machine) (s : Stage1 M) :
    (∃ x, ¬ Clean M s.e.A s.e.c s.e.k x) ∧ ¬ Escapes M s.e.A s.e.c
    s.e.k :=
  ⟨s.witnessed, fun he => s.stopped (he s.e.x)⟩

/-! ## VII · RA read on computation, the one posit, and the seatless
    lock -/

/-- RA READ ON COMPUTATION: every decider of satisfiability actuates
    beyond every polynomial bound. -/
def RAcomp (M : Machine) : Prop := ∀ A c k, ¬ Escapes M A c k

theorem ra_comp_is_sep (M : Machine) : RAcomp M ↔ Sep M :=
  ⟨fun h ⟨A, c, k, he⟩ => h A c k he, fun h A c k he => h ⟨A, c, k,
    he⟩⟩

/-- THE ONE POSIT IS THE BIT: RA read on computation is exactly the
    transfer. -/
theorem the_posit_is_the_bit (M : Machine) (x₀ : M.Inst) :
    RAcomp M ↔ (NothingEscapes (Rec M) → NothingEscapes M) :=
  ⟨fun h => (sep_normal_form M x₀).mpr ((ra_comp_is_sep M).mp h),
   fun h => (ra_comp_is_sep M).mpr ((sep_normal_form M x₀).mp h)⟩

/-- The seat of the row is vacant: complementation of worlds has no
    fixed point (PNP.VANISH, restated). -/
def comp (b : Bool) : Bool := !b
theorem locus_vanishes : ¬ ∃ b, comp b = b := fun ⟨b, h⟩ => bnot_ne b h

/-- SEATLESS: there is no fixed world to cross at, and the lock closes
    at the act on exactly the world the act
    names (the mirror of nothing in the Riemann master, where the seat
    is inhabited: the one difference). -/
theorem seatless_lock (M : Machine) :
    (¬ ∃ b, comp b = b) ∧ (Nonempty (LeastEscape M) ↔ Sep M) :=
  ⟨locus_vanishes, ⟨fun ⟨L⟩ => sep_of_closure M L.closure, fun hs =>
    ⟨(closure_of_sep M hs)⟩⟩⟩

/-! ## VIII · the capstone, nothing before the colon but the machine
    and an instance -/

/-- THE P VERSUS NP NORMAL FORM, COMPLETE. The record has least
    escape; the separation is the transfer; the
    transfer is the act; the machine is self-recursive exactly when
    separated; every route passes the bit; the one
    posit is the bit; the seat is vacant. Separation is not a
    conjunct: it occurs only inside biconditionals and
    inside the premise of a route. -/
theorem pnp_normal_form_complete (M : Machine) (x₀ : M.Inst) :
    NothingEscapes (Rec M) ∧
    ((NothingEscapes (Rec M) → NothingEscapes M) ↔ Sep M) ∧
    ((NothingEscapes (Rec M) → NothingEscapes M) ↔ Nonempty
    (LeastEscape M)) ∧
    (SelfRecursive M ↔ Sep M) ∧
    (∀ Q : Prop, (Q → Sep M) → Q → (NothingEscapes (Rec M) →
    NothingEscapes M)) ∧
    (RAcomp M ↔ (NothingEscapes (Rec M) → NothingEscapes M)) ∧
    (¬ ∃ b, comp b = b) :=
  ⟨record_has_least_escape M x₀, sep_normal_form M x₀, bit_is_the_act
    M x₀, self_recursion_at_machine M,
   fun Q h => every_route_passes_the_bit M x₀ Q h,
   the_posit_is_the_bit M x₀, locus_vanishes⟩

/-- THE TWO ROWS, ONE SHAPE, TWO BITS. The Riemann bit and the
    separation bit are each a transfer from a record
    that has the property to the object; neither follows from the
    other's record, since the stage world's record
    has least escape while the stage world escapes. The bits are two.
    -/
theorem two_rows_two_bits :
    NothingEscapes (Rec (stageWorld 0)) ∧ ¬ NothingEscapes (stageWorld
    0) :=
  ⟨record_has_least_escape (stageWorld 0) (), stage_fails_closure 0⟩

end PNP.NORMALFORM
```

```
/-! # PART B · THE SAT INSTANCE, on no axiom -/

/-! ## IX · zero-axiom helpers: lists, arithmetic, halving -/
namespace PNP.ZA

theorem append_assoc {α : Type} : ∀ (l₁ l₂ l₃ : List α), (l₁ ++ l₂) ++ l₃ = l₁ ++ (l₂ ++ l₃)
  | [], _, _ => rfl
  | a :: l₁, l₂, l₃ => congrArg (List.cons a) (append_assoc l₁ l₂ l₃)

theorem length_append {α : Type} : ∀ (l₁ l₂ : List α), (l₁ ++ l₂).length = l₁.length + l₂.length
  | [], _ => (Nat.zero_add _).symm
  | _ :: l₁, l₂ => (congrArg Nat.succ (length_append l₁ l₂)).trans (Nat.succ_add _ _).symm

theorem any_append {α : Type} (p : α → Bool) : ∀ (l₁ l₂ : List α),
    (l₁ ++ l₂).any p = (l₁.any p || l₂.any p)
  | [], _ => rfl
  | a :: l₁, l₂ => by
    show (p a || (l₁ ++ l₂).any p) = ((p a || l₁.any p) || l₂.any p)
    rw [any_append p l₁ l₂]
    cases p a <;> rfl

theorem any_flatMap {α β : Type} (p : β → Bool) (g : α → List β) : ∀ l : List α,
    (l.flatMap g).any p = l.any (fun x => (g x).any p)
  | [] => rfl
  | a :: l => by
    show (g a ++ l.flatMap g).any p = ((g a).any p || l.any (fun x => (g x).any p))
    rw [any_append, any_flatMap p g l]

theorem any_true_of {α : Type} (p : α → Bool) : ∀ l : List α, l.any p = true → ∃ x, p x = true
  | [], h => Bool.noConfusion h
  | a :: l, h => by
    cases hpa : p a with
    | true => exact ⟨a, hpa⟩
    | false =>
      have h' : l.any p = true := by
        have : (p a || l.any p) = true := h
        rw [hpa] at this
        exact this
      exact any_true_of p l h'

theorem or_true_left {a b : Bool} (h : a = true) : (a || b) = true := by
  rw [h]; rfl

theorem or_true_right {a b : Bool} (h : b = true) : (a || b) = true := by
  rw [h]; cases a <;> rfl

/-- `All P l`: P holds of every element, by recursion on the list. -/
def All {α : Type} (P : α → Prop) : List α → Prop
  | [] => True
  | x :: l => P x ∧ All P l

theorem All_append {α : Type} (P : α → Prop) : ∀ (l₁ l₂ : List α),
    All P (l₁ ++ l₂) → All P l₁ ∧ All P l₂
  | [], _, h => ⟨trivial, h⟩
  | _ :: l₁, l₂, ⟨hx, h⟩ =>
    match All_append P l₁ l₂ h with
    | ⟨h₁, h₂⟩ => ⟨⟨hx, h₁⟩, h₂⟩

theorem mul_assoc' (n m : Nat) : ∀ k : Nat, n * m * k = n * (m * k)
  | 0 => rfl
  | k+1 => by
    show n * m * k + n * m = n * (m * k + m)
    rw [Nat.left_distrib, mul_assoc' n m k]

theorem add_mul' (a b k : Nat) : (a + b) * k = a * k + b * k := by
  rw [Nat.mul_comm, Nat.left_distrib, Nat.mul_comm k a, Nat.mul_comm k b]

theorem succ_ne_zero' (n : Nat) : n + 1 ≠ 0 := fun h => Nat.noConfusion h

/-- Halving and parity, by two-step recursion; no division operator is used anywhere in the file. -/
def div2 : Nat → Nat
  | 0 => 0
  | 1 => 0
  | n+2 => div2 n + 1

def mod2 : Nat → Bool
  | 0 => false
  | 1 => true
  | n+2 => mod2 n

/-- The digit and the half recompose the number. -/
theorem digit_spec : ∀ n : Nat, (if mod2 n then 1 else 0) + 2 * div2 n = n
  | 0 => rfl
  | 1 => rfl
  | n+2 => by
    show (if mod2 n then 1 else 0) + 2 * (div2 n + 1) = n + 2
    rw [Nat.mul_succ, ← Nat.add_assoc, digit_spec n]

theorem div2_le : ∀ n : Nat, div2 n ≤ n
  | 0 => Nat.le_refl 0
  | 1 => Nat.zero_le 1
  | n+2 => Nat.succ_le_succ (Nat.le_succ_of_le (div2_le n))

theorem div2_lt : ∀ n : Nat, 0 < n → div2 n < n
  | 0, h => absurd h (Nat.lt_irrefl 0)
  | 1, _ => Nat.zero_lt_succ 0
  | n+2, _ => Nat.succ_le_succ (Nat.succ_le_succ (div2_le n))

end PNP.ZA

/-! ## X · formulas, assignments, satisfiability by exhaustion -/
namespace PNP.SAT
open PNP.NORMALFORM PNP.ZA

abbrev Lit := Nat × Bool
abbrev Clause := List Lit
abbrev CNF := List Clause
abbrev Assignment := Nat → Bool
/-- A literal (n, b) is true under a when a n = b: b = true the positive literal, b = false the negative. -/
def evalLit (a : Assignment) (l : Lit) : Bool := a l.1 == l.2
def evalClause (a : Assignment) (c : Clause) : Bool := c.any (evalLit a)
def evalCNF (a : Assignment) (f : CNF) : Bool := f.all (evalClause a)
def varsClause (c : Clause) : List Nat := c.map (·.1)
def vars : CNF → List Nat
  | [] => []
  | c :: f => varsClause c ++ vars f
def extend (v : Nat) (b : Bool) (a : Assignment) : Assignment := fun n => if n = v then b else a n
/-- Every assignment to a finite list of variables, the variables outside the list fixed false. -/
def assignments : List Nat → List Assignment
  | [] => [fun _ => false]
  | v :: vs => (assignments vs).flatMap (fun a => [extend v true a, extend v false a])
def restrict : List Nat → Assignment → Assignment
  | [], _ => fun _ => false
  | v :: vs, a => extend v (a v) (restrict vs a)
/-- THE DECISION PROCEDURE: satisfiability by exhaustion over the variables that occur. -/
def satB (f : CNF) : Bool := (assignments (vars f)).any (fun a => evalCNF a f)
/-- SATISFIABILITY, the mathematical predicate. -/
def Satisfiable (f : CNF) : Prop := ∃ a : Assignment, evalCNF a f = true
/-- The size of a formula: its clauses and its literals counted together. -/
def size : CNF → Nat
  | [] => 0
  | c :: f => c.length + 1 + size f

theorem extend_agrees (a : Assignment) (v : Nat) (r : Assignment) :
    ∀ vs' : List Nat, All (fun n => r n = a n) vs' → All (fun n => extend v (a v) r n = a n) vs'
  | [], _ => trivial
  | n :: vs', ⟨hn, h⟩ =>
    ⟨(if hv : n = v then by
        show (if n = v then a v else r n) = a n
        rw [if_pos hv, hv]
      else by
        show (if n = v then a v else r n) = a n
        rw [if_neg hv]; exact hn),
     extend_agrees a v r vs' h⟩

/-- The restriction agrees with the assignment on every listed variable. -/
theorem restrict_agrees (a : Assignment) : ∀ vs : List Nat, All (fun n => restrict vs a n = a n) vs
  | [] => trivial
  | v :: vs =>
    ⟨show (if v = v then a v else restrict vs a v) = a v from if_pos rfl,
     extend_agrees a v (restrict vs a) vs (restrict_agrees a vs)⟩

theorem evalClause_congr (a b : Assignment) :
    ∀ c : Clause, All (fun n => a n = b n) (varsClause c) → evalClause a c = evalClause b c
  | [], _ => rfl
  | l :: c, ⟨hl, h⟩ => by
    show (evalLit a l || evalClause a c) = (evalLit b l || evalClause b c)
    rw [evalClause_congr a b c h]
    show ((a l.1 == l.2) || evalClause b c) = ((b l.1 == l.2) || evalClause b c)
    rw [hl]
```

```
theorem evalCNF_congr (a b : Assignment) :
    ∀ f : CNF, All (fun n => a n = b n) (vars f) → evalCNF a f =
  evalCNF b f
  | [], _ => rfl
  | c :: f, h =>
    match All_append _ (varsClause c) (vars f) h with
    | (hc, hf) => by
      show (evalClause a c && evalCNF a f) = (evalClause b c &&
  evalCNF b f)
      rw [evalClause_congr a b c hc, evalCNF_congr a b f hf]

/-- SOUNDNESS: the procedure says satisfiable only when a satisfying
  assignment exists. -/
theorem satB_sound (f : CNF) (h : satB f = true) : Satisfiable f :=
  any_true_of (fun a => evalCNF a f) (assignments (vars f)) h

/-- The exhaustion finds any assignment through its restriction. -/
theorem any_assignments (q : Assignment → Bool) (a : Assignment) :
    ∀ vs : List Nat, q (restrict vs a) = true → (assignments vs).any q
  = true
  | [], h => or_true_left h
  | v :: vs, h => by
    show ((assignments vs).flatMap (fun b => [extend v true b, extend
  v false b])).any q = true
    rw [any_flatMap]
    apply any_assignments (fun b => [extend v true b, extend v false
  b].any q) a vs
    show (q (extend v true (restrict vs a)) || (q (extend v false
  (restrict vs a)) || false)) = true
    have h' : q (extend v (a v) (restrict vs a)) = true := h
    cases hav : a v with
    | true =>
      rw [hav] at h'
      exact or_true_left h'
    | false =>
      rw [hav] at h'
      exact or_true_right (or_true_left h')

/-- COMPLETENESS: every satisfying assignment is found, through its
  restriction to the variables that occur. -/
theorem satB_complete (f : CNF) (h : Satisfiable f) : satB f = true :=
  match h with
  | (a, ha) =>
    any_assignments (fun b => evalCNF b f) a (vars f)
      ((evalCNF_congr (restrict (vars f) a) a f (restrict_agrees a
  (vars f))).trans ha)

/-- THE PROCEDURE IS SATISFIABILITY. -/
theorem satB_iff (f : CNF) : satB f = true ↔ Satisfiable f :=
  (satB_sound f, satB_complete f)

/-! ## XI · the SAT machine over any algorithm model -/

/-- THE SAT MACHINE. An algorithm model is a type of algorithms with
  an output and a step count on every
    formula; nothing about it is assumed. The instances are the
  formulas, the size the clause-and-literal count,
    satisfiability the decided predicate. -/
def SAT (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat)
  : Machine :=
  (Alg, CNF, size, out, time, satB)

/-- The empty formula is an instance, and it is satisfiable. -/
theorem empty_satisfiable : satB [] = true := rfl

/-- SEPARATION OF SAT: no algorithm of the model decides
  satisfiability within any polynomial bound. -/
abbrev SepSAT (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF
  → Nat) : Prop := Sep (SAT Alg out time)

/-- THE RECORD OF SAT HAS LEAST ESCAPE, unconditionally, in every
  algorithm model. -/
theorem sat_record_has_least_escape (Alg : Type) (out : Alg → CNF →
  Bool) (time : Alg → CNF → Nat) :
    NothingEscapes (Rec (SAT Alg out time)) :=
  record_has_least_escape (SAT Alg out time) []

/-- THE NORMAL FORM OF SAT: the separation is exactly the transfer of
  least escape from the record of SAT to SAT,
    and the antecedent is a theorem, so the separation is one
  implication. -/
theorem sat_normal_form (Alg : Type) (out : Alg → CNF → Bool) (time :
  Alg → CNF → Nat) :
    (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg
  out time)) ↔ SepSAT Alg out time :=
  sep_normal_form (SAT Alg out time) []

/-- The transfer is the act on SAT. -/
theorem sat_bit_is_the_act (Alg : Type) (out : Alg → CNF → Bool) (time
  : Alg → CNF → Nat) :
    (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg
  out time)) ↔
      Nonempty (LeastEscape (SAT Alg out time)) :=
  bit_is_the_act (SAT Alg out time) []

/-- SAT is self-recursive exactly when separated; its record is
  self-recursive with no premise. -/
theorem sat_self_recursion (Alg : Type) (out : Alg → CNF → Bool) (time
  : Alg → CNF → Nat) :
    (SelfRecursive (SAT Alg out time) ↔ SepSAT Alg out time) ∧
  SelfRecursive (Rec (SAT Alg out time)) :=
  (self_recursion_at_machine (SAT Alg out time),
  record_is_self_recursive (SAT Alg out time) [])

/-- EVERY ROUTE TO THE SEPARATION OF SAT PASSES THE BIT. -/
theorem sat_every_route_passes_the_bit (Alg : Type) (out : Alg → CNF →
  Bool) (time : Alg → CNF → Nat)
    (Q : Prop) (h : Q → SepSAT Alg out time) :
    Q → (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT
  Alg out time)) :=
  every_route_passes_the_bit (SAT Alg out time) [] Q h

/-- RA READ ON COMPUTATION IS THE BIT, on SAT. -/
theorem sat_posit_is_the_bit (Alg : Type) (out : Alg → CNF → Bool)
  (time : Alg → CNF → Nat) :
    RAcomp (SAT Alg out time) ↔ (NothingEscapes (Rec (SAT Alg out
  time)) → NothingEscapes (SAT Alg out time)) :=
  the_posit_is_the_bit (SAT Alg out time) []

/-- THE SAT NORMAL FORM, COMPLETE: the capstone of Part A on the SAT
  machine, in every algorithm model. -/
theorem sat_normal_form_complete (Alg : Type) (out : Alg → CNF → Bool)
  (time : Alg → CNF → Nat) :
    NothingEscapes (Rec (SAT Alg out time)) ∧
    ((NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT
  Alg out time)) ↔ SepSAT Alg out time) ∧
    ((NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT
  Alg out time)) ↔
      Nonempty (LeastEscape (SAT Alg out time))) ∧
    (SelfRecursive (SAT Alg out time) ↔ SepSAT Alg out time) ∧
    (∀ Q : Prop, (Q → SepSAT Alg out time) →
      Q → (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes
  (SAT Alg out time))) ∧
    (RAcomp (SAT Alg out time) ↔ (NothingEscapes (Rec (SAT Alg out
  time)) → NothingEscapes (SAT Alg out time))) ∧
    (¬ ∃ b, comp b = b) :=
  pnp_normal_form_complete (SAT Alg out time) []

/-! ## XII · the SAT-fixed worlds: the wall on the formulas themselves
  -/

/-- A closed world of SAT: every algorithm answers false in no time;
  nothing escapes, since the empty formula
    is satisfiable. Instances CNF, size, predicate satB, all fixed. -/
def satClosedWorld : Machine := SAT Nat (fun _ _ => false) (fun _ _ =>
  0)
/-- A stage world of SAT: algorithm n answers satisfiability itself in
  no time and escapes; the others answer
    false. The same instances, size and predicate. -/
def satStageWorld (n : Nat) : Machine :=
  SAT Nat (fun A f => if Nat.beq A n then satB f else false) (fun _ _
  => 0)

theorem sat_closed_world_closes : NothingEscapes satClosedWorld :=
  fun _ _ _ => ([], fun (h, _) => Bool.noConfusion h)

theorem sat_stage_escapes (n : Nat) : Escapes (satStageWorld n) n 0 0
  :=
  fun f => (show (if Nat.beq n n then satB f else false) = satB f by
  rw [beq_self n]; rfl, Nat.zero_le _)

theorem sat_stage_fails_closure (n : Nat) : ¬ NothingEscapes
  (satStageWorld n) :=
  fun h => sep_of_closure _ h (n, 0, 0, sat_stage_escapes n)

/-- THE BIT IS KEYED ON SAT ITSELF: the transfer holds on the closed
  SAT world and fails on the stage SAT world,
    both deciding the same formulas with the same predicate. -/
theorem sat_bit_is_keyed :
    (NothingEscapes (Rec satClosedWorld) → NothingEscapes
  satClosedWorld) ∧
    ¬ (NothingEscapes (Rec (satStageWorld 0)) → NothingEscapes
  (satStageWorld 0)) :=
  (fun _ => sat_closed_world_closes,
   fun h => sat_stage_fails_closure 0 (h (record_has_least_escape
  (satStageWorld 0) [])))

/-- A property of algorithm models of SAT: it ranges over the machines
  `SAT Alg out time` and nothing else. -/
def SATKeyless (P : Machine → Prop) : Prop :=
  ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat), P
  (SAT Alg out time)
def SATForces (P : Machine → Prop) : Prop :=
  ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
    P (SAT Alg out time) → NothingEscapes (SAT Alg out time)

/-- THE KEYLESS WALL ON SAT: no property shared by every algorithm
```

```
     model of SAT forces the closure of any. -/
552 theorem sat_keyless_forces_nothing (P : Machine → Prop) (hP :
     SATKeyless P) : ¬ SATForces P :=
553   fun h => sat_stage_fails_closure 0 (h Nat _ _ (hP Nat _ _))
554
555 /-- NO RECORD PRINCIPLE DECIDES SAT: a principle true of the record of
     every algorithm model of SAT decides
556     none of them. -/
557 theorem sat_no_record_principle_decides (P : Machine → Prop)
558     (hP : ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF →
     Nat), P (Rec (SAT Alg out time))) :
559     ¬ ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
560         P (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)
     :=
561   fun h => sat_stage_fails_closure 0 (h Nat _ _ (hP Nat _ _))
562
563 /-- THE RECORD DECIDES NOTHING ON SAT: both SAT worlds' records close;
     one world closes and the other escapes. -/
564 theorem sat_record_decides_nothing :
565     NothingEscapes (Rec satClosedWorld) ∧ NothingEscapes (Rec
     (satStageWorld 0)) ∧
566     NothingEscapes satClosedWorld ∧ ¬ NothingEscapes (satStageWorld 0)
     :=
567   (record_has_least_escape satClosedWorld [], record_has_least_escape
     (satStageWorld 0) [],
568    sat_closed_world_closes, sat_stage_fails_closure 0)
569
570 /-- THE PREMISE TRICHOTOMY ON SAT (classical): a premise on algorithm
     models of SAT either has a world where it
571     holds and an algorithm escapes, or it forces the closure on every
     model it holds of; and if it forces, either
572     the separation returns it on every model, so it is the separation,
     or on some separated model it fails, so it
573     is strictly stronger than the separation. -/
574 theorem sat_premise_trichotomy (P : Machine → Prop) :
575     (∃ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
576         P (SAT Alg out time) ∧ ¬ NothingEscapes (SAT Alg out time)) ∨
577     (SATForces P ∧
578       ((∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF →
     Nat),
579           SepSAT Alg out time → P (SAT Alg out time)) ∨
580        (∃ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF →
     Nat),
581           SepSAT Alg out time ∧ ¬ P (SAT Alg out time)))) :=
582   Classical.byCases
583     (fun hF : SATForces P =>
584       Or.inr (hF, Classical.byCases
585         (fun hR : ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg
     → CNF → Nat),
586             SepSAT Alg out time → P (SAT Alg out time) => Or.inl hR)
587         (fun hR => Or.inr (Classical.byContradiction fun hne =>
588           hR (fun Alg out time hs => Classical.byContradiction fun hp
     => hne (Alg, out, time, hs, hp))))))
589     (fun hF => Or.inl (Classical.byContradiction fun hne =>
590       hF (fun Alg out time hp => Classical.byContradiction fun hc =>
     hne (Alg, out, time, hp, hc))))
591
592 end PNP.SAT
593
594 /-! # PART C · THE TURING MODEL: real machines deciding real SAT, on
     no axiom -/
595 namespace PNP.TM
596 open PNP.NORMALFORM PNP.SAT PNP.ZA
597
598 /-! ## XIII · the input alphabet and the binary encoding of formulas,
     with its decoder -/
599
600 /-- The input alphabet: two binary digits and a separator. -/
601 inductive Sym | zero | one | sep
602   deriving DecidableEq
603
604 def symOfBool : Bool → Sym
605   | true => Sym.one
606   | false => Sym.zero
607
608 theorem symOfBool_ne_sep (b : Bool) : symOfBool b ≠ Sym.sep := by
609   cases b <;> exact fun h => Sym.noConfusion h
610
611 /-- The binary digits of n, least significant first, computed with a
     fuel of halvings; `bits 0 = []`. -/
612 def bitsF : Nat → Nat → List Bool
613   | 0, _ => []
614   | fuel+1, n => if n = 0 then [] else mod2 n :: bitsF fuel (div2 n)
615
616 def bits (n : Nat) : List Bool := bitsF n n
617
618 def ofBits : List Bool → Nat
619   | [] => 0
620   | b :: l => (if b then 1 else 0) + 2 * ofBits l
621
622 theorem ofBits_bitsF : ∀ (fuel n : Nat), n ≤ fuel → ofBits (bitsF fuel
     n) = n
623   | 0, n, h => by
624     have : n = 0 := Nat.le_zero.mp h
625     subst this; rfl
626   | fuel+1, n, h =>
627     if h0 : n = 0 then by
628       show ofBits (if n = 0 then [] else mod2 n :: bitsF fuel (div2
     n)) = n
629       rw [if_pos h0, h0]; rfl
630     else by
631       show ofBits (if n = 0 then [] else mod2 n :: bitsF fuel (div2
     n)) = n
632       rw [if_neg h0]
633       show (if mod2 n then 1 else 0) + 2 * ofBits (bitsF fuel (div2
     n)) = n
634       have hlt : div2 n < n := div2_lt n (Nat.pos_of_ne_zero h0)
635       rw [ofBits_bitsF fuel (div2 n) (Nat.le_of_lt_succ
     (Nat.lt_of_lt_of_le hlt h))]
636       exact digit_spec n
637
638 theorem ofBits_bits (n : Nat) : ofBits (bits n) = n := ofBits_bitsF n
     n (Nat.le_refl n)
639
640 theorem bits_succ (n : Nat) : bits (n+1) = mod2 (n+1) :: bitsF n (div2
     (n+1)) := by
641   show (if n + 1 = 0 then [] else mod2 (n+1) :: bitsF n (div2 (n+1)))
     = _
642   rw [if_neg (succ_ne_zero' n)]
643
644 /-- Read binary digits up to the first separator: the digits, and the
     remainder after the separator. -/
645 def parseBits : List Sym → Option (List Bool × List Sym)
646   | [] => none
647   | Sym.sep :: rest => some ([], rest)
648   | Sym.zero :: rest => (parseBits rest).map (fun p => (false :: p.1,
     p.2))
649   | Sym.one :: rest => (parseBits rest).map (fun p => (true :: p.1,
     p.2))
650
651 theorem parseBits_enc (rest : List Sym) : ∀ l : List Bool,
652     parseBits (l.map symOfBool ++ Sym.sep :: rest) = some (l, rest)
653   | [] => rfl
654   | true :: l => by
655     show (parseBits (l.map symOfBool ++ Sym.sep :: rest)).map (fun p
     => (true :: p.1, p.2)) = some (true :: l, rest)
656     exact congrArg (Option.map (fun p => (true :: p.1, p.2)))
     (parseBits_enc rest l)
657   | false :: l => by
658     show (parseBits (l.map symOfBool ++ Sym.sep :: rest)).map (fun p
     => (false :: p.1, p.2)) = some (false :: l, rest)
659     exact congrArg (Option.map (fun p => (false :: p.1, p.2)))
     (parseBits_enc rest l)
660
661 /-- A number n is written as the digits of n+1 (never empty) followed
     by a separator. -/
662 def encNat (n : Nat) : List Sym := (bits (n+1)).map symOfBool ++
     [Sym.sep]
663 /-- A literal is its variable followed by its sign. -/
664 def encLit (l : Lit) : List Sym := encNat l.1 ++ [symOfBool l.2]
665 /-- A clause is its literals followed by a separator. -/
666 def encClause : Clause → List Sym
667   | [] => [Sym.sep]
668   | l :: c => encLit l ++ encClause c
669 /-- A formula is its clauses, one after the other. -/
670 def encode : CNF → List Sym
671   | [] => []
672   | c :: f => encClause c ++ encode f
673
674 def parseLit (w : List Sym) : Option (Lit × List Sym) :=
675   match parseBits w with
676   | none => none
677   | some (ds, rest) =>
678     match rest with
679     | [] => none
680     | Sym.sep :: _ => none
681     | Sym.zero :: rest' => some ((ofBits ds - 1, false), rest')
682     | Sym.one :: rest' => some ((ofBits ds - 1, true), rest')
683
684 theorem encLit_append (n : Nat) (b : Bool) (rest : List Sym) :
685     encLit (n, b) ++ rest = (bits (n+1)).map symOfBool ++ Sym.sep ::
     (symOfBool b :: rest) := by
686   show (((bits (n+1)).map symOfBool ++ [Sym.sep]) ++ [symOfBool b]) ++
     rest = _
687   rw [append_assoc, append_assoc]; rfl
688
689 theorem parseLit_enc (l : Lit) (rest : List Sym) : parseLit (encLit l
     ++ rest) = some (l, rest) := by
690   obtain (n, b) := l
691   rw [encLit_append]
692   unfold parseLit
693   rw [parseBits_enc]
694   cases b with
695   | true => exact congrArg (fun m => some ((m - 1, true), rest))
     (ofBits_bits (n+1))
696   | false => exact congrArg (fun m => some ((m - 1, false), rest))
     (ofBits_bits (n+1))
697
```

```
theorem encLit_cons (l : Lit) : ∃ s tl, encLit l = s :: tl ∧ s ≠ Sym.sep := by
  obtain ⟨n, b⟩ := l
  show ∃ s tl, ((bits (n+1)).map symOfBool ++ [Sym.sep]) ++ [symOfBool b] = s :: tl ∧ s ≠ Sym.sep
  rw [bits_succ]
  exact ⟨_, _, rfl, symOfBool_ne_sep _⟩

/-- Read a clause: literals until a separator stands where a literal would start. Fuel bounds the recursion. -/
def parseClause : Nat → List Sym → Option (Clause × List Sym)
  | 0, _ => none
  | fuel+1, w =>
    match w with
    | Sym.sep :: rest => some ([], rest)
    | _ =>
      match parseLit w with
      | none => none
      | some (l, rest) =>
        match parseClause fuel rest with
        | none => none
        | some (c, rest') => some (l :: c, rest')

theorem encClause_cons_append (l : Lit) (c : Clause) (rest : List Sym) :
    encClause (l :: c) ++ rest = encLit l ++ (encClause c ++ rest) :=
  append_assoc _ _ _

theorem parseClause_enc : ∀ (c : Clause) (fuel : Nat) (rest : List Sym), c.length ≤ fuel →
    parseClause (fuel+1) (encClause c ++ rest) = some (c, rest)
  | [], _, _, _ => rfl
  | l :: c, fuel'+1, rest, h => by
    obtain ⟨s, tl, he, hs⟩ := encLit_cons l
    have hc : c.length ≤ fuel' := Nat.le_of_succ_le_succ h
    have hw : encClause (l :: c) ++ rest = s :: (tl ++ (encClause c ++ rest)) := by
      rw [encClause_cons_append, he]; rfl
    rw [hw, parseClause]
    · rw [← List.cons_append, ← he, parseLit_enc]
      show (match parseClause (fuel'+1) (encClause c ++ rest) with
            | none => none
            | some (c', rest') => some (l :: c', rest')) = some (l :: c, rest)
      rw [parseClause_enc c fuel' rest hc]
    · intro r hr
      exact hs (List.cons.inj hr).1

theorem encLit_length_pos (l : Lit) : 1 ≤ (encLit l).length := by
  obtain ⟨s, tl, he, _⟩ := encLit_cons l
  rw [he]; exact Nat.succ_le_succ (Nat.zero_le _)

theorem encClause_length : ∀ c : Clause, c.length + 1 ≤ (encClause c).length
  | [] => Nat.le_refl 1
  | l :: c => by
    show c.length + 1 + 1 ≤ (encLit l ++ encClause c).length
    rw [length_append, Nat.add_comm (c.length + 1) 1]
    exact Nat.add_le_add (encLit_length_pos l) (encClause_length c)

theorem encClause_cons (c : Clause) : ∃ s tl, encClause c = s :: tl := by
  cases c with
  | nil => exact ⟨Sym.sep, [], rfl⟩
  | cons l c =>
    obtain ⟨s, tl, he, _⟩ := encLit_cons l
    exact ⟨s, tl ++ encClause c, by show encLit l ++ encClause c = _; rw [he]; rfl⟩

theorem encClause_ne_nil (c : Clause) (rest : List Sym) : encClause c ++ rest ≠ [] := by
  obtain ⟨s, tl, he⟩ := encClause_cons c
  rw [he]; exact fun h => List.noConfusion h

/-- Read a formula: clauses until the string ends. Fuel bounds the recursion. -/
def parseCNF : Nat → List Sym → Option CNF
  | _, [] => some []
  | 0, _ :: _ => none
  | fuel+1, x :: w =>
    match parseClause (x :: w).length (x :: w) with
    | none => none
    | some (c, rest) =>
      match parseCNF fuel rest with
      | none => none
      | some f => some (c :: f)

/-- THE DECODER. -/
def decode (w : List Sym) : Option CNF := parseCNF w.length w

theorem encode_length : ∀ f : CNF, f.length ≤ (encode f).length
  | [] => Nat.le_refl 0
  | c :: f => by
    show f.length + 1 ≤ (encClause c ++ encode f).length
    rw [length_append, Nat.add_comm f.length 1]
    exact Nat.add_le_add (Nat.le_trans (Nat.le_add_left 1 c.length) (encClause_length c)) (encode_length f)

theorem parseCNF_enc : ∀ (f : CNF) (fuel : Nat), f.length ≤ fuel →
    parseCNF fuel (encode f) = some f
  | [], fuel, _ => by cases fuel <;> rfl
  | c :: f, fuel'+1, h => by
    have hf : f.length ≤ fuel' := Nat.le_of_succ_le_succ h
    show parseCNF (fuel'+1) (encClause c ++ encode f) = some (c :: f)
    cases hE : encClause c ++ encode f with
    | nil => exact absurd hE (encClause_ne_nil c _)
    | cons x w =>
      have hcw : c.length ≤ w.length := by
        have h1 : (encClause c ++ encode f).length = w.length + 1 := congrArg List.length hE
        rw [length_append] at h1
        exact Nat.le_of_succ_le_succ (Nat.le_trans (encClause_length c)
          (Nat.le_trans (Nat.le_add_right _ _) (Nat.le_of_eq h1)))
      show (match parseClause (w.length + 1) (x :: w) with
            | none => none
            | some (c', rest) =>
              match parseCNF fuel' rest with
              | none => none
              | some f' => some (c' :: f')) = some (c :: f)
      rw [← hE, parseClause_enc c w.length (encode f) hcw]
      show (match parseCNF fuel' (encode f) with
            | none => none
            | some f' => some (c :: f')) = some (c :: f)
      rw [parseCNF_enc f fuel' hf]

/-- THE DECODER INVERTS THE ENCODING. -/
theorem decode_encode (f : CNF) : decode (encode f) = some f :=
  parseCNF_enc f _ (encode_length f)

/-- THE ENCODING IS INJECTIVE: distinct formulas have distinct strings. -/
theorem encode_injective (f g : CNF) (h : encode f = encode g) : f = g := by
  have hf := decode_encode f
  rw [h, decode_encode] at hf
  exact (Option.some.inj hf).symm

/-- SAT AS A LANGUAGE over the input alphabet: a string is in SAT when it decodes to a satisfiable formula. -/
def satLang (w : List Sym) : Bool :=
  match decode w with
  | some f => satB f
  | none => false

theorem satLang_encode (f : CNF) : satLang (encode f) = satB f := by
  unfold satLang
  rw [decode_encode]

theorem satLang_encode_iff (f : CNF) : satLang (encode f) = true ↔ Satisfiable f := by
  rw [satLang_encode]; exact satB_iff f

/-! ## XIV · Turing machines: finite control, one two-way tape, step semantics -/

inductive Dir | L | R

/-- A TURING MACHINE. States are `Fin q`; tape symbols are `Fin (s + 4)`: 0 the blank, 1 and 2 the binary
    digits, 3 the separator, and s further work symbols. The transition function is total; the machine halts in
    its accept or reject state. -/
structure TM where
  q : Nat
  s : Nat
  δ : Fin q → Fin (s + 4) → Fin q × Fin (s + 4) × Dir
  start : Fin q
  accept : Fin q
  reject : Fin q

theorem lt_add4 (i s : Nat) (h : i < 4) : i < s + 4 :=
  Nat.lt_of_lt_of_le h (Nat.le_add_left 4 s)

def Sym.toTape (s : Nat) : Sym → Fin (s + 4)
  | Sym.zero => ⟨1, lt_add4 1 s (by decide)⟩
  | Sym.one => ⟨2, lt_add4 2 s (by decide)⟩
  | Sym.sep => ⟨3, lt_add4 3 s (by decide)⟩

structure Config (M : TM) where
  st : Fin M.q
  pos : Int
  tape : Int → Fin (M.s + 4)

def blank (M : TM) : Fin (M.s + 4) := ⟨0, lt_add4 0 M.s (by decide)⟩

/-- The input written from cell 0 rightward, blank elsewhere. -/
def initTape (M : TM) (w : List Sym) : Int → Fin (M.s + 4) :=
  fun i => if i < 0 then blank M else (w.map (Sym.toTape M.s)).getD
```

```
    i.toNat (blank M)
863
864 def init (M : TM) (w : List Sym) : Config M := (M.start, 0, initTape M
    w)
865
866 def halted (M : TM) (c : Config M) : Prop := c.st = M.accept ∨ c.st =
    M.reject
867 instance (M : TM) (c : Config M) : Decidable (halted M c) :=
    inferInstanceAs (Decidable (_ ∨ _))
868
869 def move : Dir → Int
870   | Dir.L => -1
871   | Dir.R => 1
872
873 /-- ONE STEP: read the scanned cell, write, move, change state; a
    halted configuration is fixed. -/
874 def step (M : TM) (c : Config M) : Config M :=
875   if halted M c then c else
876   let r := M.δ c.st (c.tape c.pos)
877   { st := r.1, pos := c.pos + move r.2.2, tape := fun i => if i =
    c.pos then r.2.1 else c.tape i }
878
879 /-- THE RUN: the configuration after t steps on input w. -/
880 def run (M : TM) (w : List Sym) : Nat → Config M
881   | 0 => init M w
882   | t+1 => step M (run M w t)
883
884 def HaltsAt (M : TM) (w : List Sym) (t : Nat) : Prop := halted M (run
    M w t)
885 instance (M : TM) (w : List Sym) (t : Nat) : Decidable (HaltsAt M w t)
    :=
886   inferInstanceAs (Decidable (halted M (run M w t)))
887
888 /-- The verdict after t steps: in the accept state or not. -/
889 def verdictAt (M : TM) (w : List Sym) (t : Nat) : Bool := decide ((run
    M w t).st = M.accept)
890
891 theorem step_halted (M : TM) (c : Config M) (h : halted M c) : step M
    c = c := by
892   unfold step; exact if_pos h
893
894 /-- A HALTED MACHINE STAYS WHERE IT IS: the run is constant from the
    halting step on. -/
895 theorem run_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt M w
    t) (hle : t ≤ u) :
896     run M w u = run M w t := by
897   induction hle with
898   | refl => rfl
899   | step _ ih => exact (congrArg (step M) ih).trans (step_halted M _ h)
900
901 /-- A HALTED MACHINE STAYS HALTED. -/
902 theorem haltsAt_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt
    M w t) (hle : t ≤ u) :
903     HaltsAt M w u :=
904   Eq.mpr (congrArg (halted M) (run_of_le h hle)) h
905
906 /-- THE VERDICT IS FIXED FROM THE HALTING STEP ON. -/
907 theorem verdict_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt
    M w t) (hle : t ≤ u) :
908     verdictAt M w u = verdictAt M w t :=
909   congrArg (fun c : Config M => decide (c.st = M.accept)) (run_of_le h
    hle)
910
911 /-! ## XV · the running time: the first halting step -/
912
913 /-- The first halting step at or below n, or n when none is found
    below it. -/
914 def firstHalt (M : TM) (w : List Sym) : Nat → Nat
915   | 0 => 0
916   | n+1 => if HaltsAt M w (firstHalt M w n) then firstHalt M w n else
    n+1
917
918 theorem firstHalt_le (M : TM) (w : List Sym) : ∀ n, firstHalt M w n ≤ n
919   | 0 => Nat.le_refl 0
920   | n+1 =>
921     if h : HaltsAt M w (firstHalt M w n) then by
922       show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else
    n+1) ≤ n+1
923       rw [if_pos h]; exact Nat.le_succ_of_le (firstHalt_le M w n)
924     else by
925       show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else
    n+1) ≤ n+1
926       rw [if_neg h]; exact Nat.le_refl _
927
928 theorem firstHalt_halts (M : TM) (w : List Sym) : ∀ n, HaltsAt M w n →
    HaltsAt M w (firstHalt M w n)
929   | 0, h => h
930   | n+1, h =>
931     if hn : HaltsAt M w (firstHalt M w n) then by
932       show HaltsAt M w (if HaltsAt M w (firstHalt M w n) then
    firstHalt M w n else n+1)
933       rw [if_pos hn]; exact hn
934     else by
935       show HaltsAt M w (if HaltsAt M w (firstHalt M w n) then
    firstHalt M w n else n+1)
936       rw [if_neg hn]; exact h
937
938 theorem firstHalt_least (M : TM) (w : List Sym) :
939     ∀ n t, HaltsAt M w t → t ≤ n → firstHalt M w n ≤ t
940   | 0, t, _, _ => Nat.zero_le t
941   | n+1, t, h, ht =>
942     if hn : HaltsAt M w (firstHalt M w n) then by
943       show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else
    n+1) ≤ t
944       rw [if_pos hn]
945       exact match Nat.lt_or_ge t (n+1) with
946         | Or.inl hlt => firstHalt_least M w n t h (Nat.le_of_lt_succ
    hlt)
947         | Or.inr hge => Nat.le_trans (firstHalt_le M w n)
    (Nat.le_trans (Nat.le_succ n) hge)
948     else by
949       show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else
    n+1) ≤ t
950       rw [if_neg hn]
951       exact match Nat.lt_or_ge t (n+1) with
952         | Or.inl hlt => absurd (firstHalt_halts M w n (haltsAt_of_le h
    (Nat.le_of_lt_succ hlt))) hn
953         | Or.inr hge => hge
954
955 /-- A DECIDER: a Turing machine with a clock that certifies it halts
    on every input. The clock certifies and
956     does not time: the running time below is the first halting step,
    whatever the clock says. -/
957 structure Decider where
958   M : TM
959   clock : List Sym → Nat
960   halts : ∀ w : List Sym, HaltsAt M w (clock w)
961
962 /-- THE RUNNING TIME: the first halting step. -/
963 def time (D : Decider) (w : List Sym) : Nat := firstHalt D.M w
    (D.clock w)
964 /-- THE OUTPUT: the verdict at the first halting step. -/
965 def out (D : Decider) (w : List Sym) : Bool := verdictAt D.M w (time D
    w)
966
967 theorem time_halts (D : Decider) (w : List Sym) : HaltsAt D.M w (time
    D w) :=
968   firstHalt_halts D.M w (D.clock w) (D.halts w)
969
970 theorem time_least (D : Decider) (w : List Sym) (t : Nat) (h : HaltsAt
    D.M w t) : time D w ≤ t :=
971   match Nat.lt_or_ge t (D.clock w) with
972   | Or.inl hlt => firstHalt_least D.M w (D.clock w) t h (Nat.le_of_lt
    hlt)
973   | Or.inr hge => Nat.le_trans (firstHalt_le D.M w (D.clock w)) hge
974
975 theorem out_eq (D : Decider) (w : List Sym) (t : Nat) (h : HaltsAt D.M
    w t) : out D w = verdictAt D.M w t :=
976   (verdict_of_le (time_halts D w) (time_least D w t h)).symm
977
978 /-! ## XVI · the SAT machine of the Turing model, and SAT ∉ P stated
    over all Turing machines -/
979
980 /-- THE SAT MACHINE OF THE TURING MODEL. Algorithms: deciders.
    Instances: strings over the input alphabet.
981     Size: the length of the string. Output and time: the verdict and
    the step count of the run. Predicate: SAT. -/
982 def TMSAT : Machine := (Decider, List Sym, List.length, out, time,
    satLang)
983
984 /-- A language over the input alphabet. -/
985 abbrev Lang := List Sym → Bool
986
987 /-- A LANGUAGE IS IN P: some Turing machine halts on every input
    within c·n^k + c steps, n the length of the
988     input, in the accept state exactly on the strings of the language.
    -/
989 def InP (L : Lang) : Prop := ∃ (M : TM) (c k : Nat), ∀ w : List Sym,
990   ∃ t, t ≤ bound c k w.length ∧ HaltsAt M w t ∧ verdictAt M w t = L w
991
992 /-- SAT ∈ P, the proposition, over every Turing machine. -/
993 def SATinP : Prop := InP satLang
994
995 theorem satInP_iff : SATinP ↔ InP satLang := Iff.rfl
996
997 /-- A LANGUAGE IS IN NP: some Turing machine, the verifier, accepts
    within c·n^k + c steps the string made of a
998     binary certificate of length at most c·n^k + c, the separator, and
    the input, exactly when the input is in the
999     language; n is the length of the input. The certificate contains
    no separator, so the first separator on the
1000     tape delimits it. -/
1001 def InNP (L : Lang) : Prop := ∃ (M : TM) (c k : Nat), ∀ w : List Sym,
1002   (L w = true ↔ ∃ u : List Bool, u.length ≤ bound c k w.length ∧
1003     ∃ t, t ≤ bound c k w.length ∧ HaltsAt M (u.map symOfBool ++
    Sym.sep :: w) t ∧
1004       verdictAt M (u.map symOfBool ++ Sym.sep :: w) t = true)
1005
```

```
/-- P ≠ NP, the sentence: some language is in NP and not in P. -/
def P_ne_NP : Prop := ∃ L : Lang, InNP L ∧ ¬ InP L

/-- THE ONE-WAY BRIDGE, with SAT ∈ NP as its named hypothesis: SAT ∉ P gives P ≠ NP. -/
theorem p_ne_np_of_not_inP (hNP : InNP satLang) (h : ¬ SATinP) : P_ne_NP := ⟨satLang, hNP, h⟩

/-- THE BRIDGE: the separation of the SAT machine is exactly SAT ∉ P over all Turing machines. -/
theorem sep_tmsat_iff : Sep TMSAT ↔ ¬ SATinP := by
  constructor
  · intro hsep ⟨M, c, k, h⟩
    let D : Decider := ⟨M, fun w => bound c k w.length,
      fun w => match h w with | ⟨t, hle, ht, _⟩ => haltsAt_of_le ht hle⟩
    apply hsep
    refine ⟨D, c, k, fun w => ?_⟩
    obtain ⟨t, hle, ht, hv⟩ := h w
    exact ⟨(out_eq D w t ht).trans hv, Nat.le_trans (time_least D w t ht) hle⟩
  · intro hnot ⟨D, c, k, he⟩
    apply hnot
    refine ⟨D.M, c, k, fun w => ?_⟩
    obtain ⟨hout, htime⟩ := he w
    exact ⟨time D w, htime, time_halts D w, hout⟩

/-- Every natural-coefficient polynomial is dominated by a bound of the kernel's form. -/
def evalPoly : List Nat → Nat → Nat
  | [], _ => 0
  | a :: as, n => a + n * evalPoly as n

theorem evalPoly_le (as : List Nat) (n : Nat) (hn : 1 ≤ n) : evalPoly as n ≤ as.sum * n ^ as.length := by
  induction as with
  | nil => exact Nat.le_refl 0
  | cons a as ih =>
    show a + n * evalPoly as n ≤ (a + as.sum) * (n ^ as.length * n)
    have hpos : 1 ≤ n ^ as.length * n := Nat.mul_pos (Nat.pow_pos hn) hn
    have h1 : a ≤ a * (n ^ as.length * n) :=
      Nat.le_trans (Nat.le_of_eq (Nat.mul_one a).symm) (Nat.mul_le_mul_left a hpos)
    have h2 : n * evalPoly as n ≤ as.sum * (n ^ as.length * n) := by
      have := Nat.mul_le_mul_left n ih
      rw [Nat.mul_comm n (as.sum * n ^ as.length), mul_assoc'] at this
      exact this
    rw [add_mul']
    exact Nat.add_le_add h1 h2

theorem evalPoly_le_bound (as : List Nat) (n : Nat) : evalPoly as n ≤ bound as.sum as.length n :=
  match n with
  | 0 => by
    cases as with
    | nil => exact Nat.le_refl 0
    | cons a as =>
      show a + 0 * evalPoly as 0 ≤ (a + as.sum) * 0 ^ (as.length + 1) + (a + as.sum)
      rw [Nat.zero_mul, Nat.add_zero, Nat.pow_succ, Nat.mul_zero, Nat.mul_zero, Nat.zero_add]
      exact Nat.le_add_right a as.sum
  | n+1 => Nat.le_trans (evalPoly_le as (n+1) (Nat.succ_le_succ (Nat.zero_le n))) (Nat.le_add_right _ _)

/-- SAT ∈ P in the polynomial's own terms implies SAT ∈ P in the kernel's: nothing is lost in the bound. -/
theorem SATinP_of_poly (M : TM) (as : List Nat)
    (h : ∀ w : List Sym, ∃ t, t ≤ evalPoly as w.length ∧ HaltsAt M w t ∧ verdictAt M w t = satLang w) :
    SATinP :=
  ⟨M, as.sum, as.length, fun w => match h w with
    | ⟨t, hle, ht, hv⟩ => ⟨t, Nat.le_trans hle (evalPoly_le_bound as w.length), ht, hv⟩⟩

/-! ## XVII · the normal form on the Turing model -/

theorem tmsat_record_has_least_escape : NothingEscapes (Rec TMSAT) :=
  record_has_least_escape TMSAT []

/-- THE NORMAL FORM OF P VERSUS NP ON THE TURING MODEL: SAT ∉ P is exactly the transfer of least escape from the
    record of the SAT machine to the SAT machine. -/
theorem pnp_normal_form_turing :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ ¬ SATinP :=
  (sep_normal_form TMSAT []).trans sep_tmsat_iff

theorem tmsat_bit_is_the_act :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ Nonempty (LeastEscape TMSAT) :=
  bit_is_the_act TMSAT []

theorem tmsat_self_recursion : (SelfRecursive TMSAT ↔ ¬ SATinP) ∧
    SelfRecursive (Rec TMSAT) :=
  ⟨(self_recursion_at_machine TMSAT).trans sep_tmsat_iff, record_is_self_recursive TMSAT []⟩

/-- EVERY ROUTE TO SAT ∉ P PASSES THE BIT. -/
theorem tmsat_every_route_passes_the_bit (Q : Prop) (h : Q → ¬ SATinP) :
    Q → (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) :=
  every_route_passes_the_bit TMSAT [] Q (fun q => sep_tmsat_iff.mpr (h q))

theorem tmsat_posit_is_the_bit :
    RAcomp TMSAT ↔ (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) :=
  the_posit_is_the_bit TMSAT []

/-- THE BRIDGE TEST: the register's reading of the root on computation, every decider at every bound failing to
    escape, is exactly SAT ∉ P on the Turing model, on no axiom. -/
theorem tmsat_racomp_iff_not_inP : RAcomp TMSAT ↔ ¬ SATinP :=
  (ra_comp_is_sep TMSAT).trans sep_tmsat_iff

/-- EVERY EXIT IS THE POSIT: any proof of SAT ∉ P, by any route, including one built from facts about Turing
    machines, yields all three names of the one posit: the transfer, a least-escape certificate, and the root
    read on computation. -/
theorem every_exit_is_the_posit (h : ¬ SATinP) :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ∧ Nonempty (LeastEscape TMSAT) ∧ RAcomp TMSAT :=
  have bit : NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT := pnp_normal_form_turing.mpr h
  ⟨bit, tmsat_bit_is_the_act.mp bit, tmsat_racomp_iff_not_inP.mpr h⟩

/-- THE POSIT IS EVERY EXIT: each name of the posit gives SAT ∉ P back, on no axiom. Nothing stands between the
    posit and the conclusion. -/
theorem the_posit_is_every_exit :
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) → ¬ SATinP) ∧
    (Nonempty (LeastEscape TMSAT) → ¬ SATinP) ∧
    (RAcomp TMSAT → ¬ SATinP) :=
  ⟨fun bit => sep_tmsat_iff.mp (sep_from_the_cargo_loop TMSAT [] bit),
   fun ⟨L⟩ => sep_tmsat_iff.mp (sep_of_closure TMSAT L.closure),
   fun h => tmsat_racomp_iff_not_inP.mp h⟩

/-- THE CAPSTONE ON THE TURING MODEL. -/
theorem tmsat_normal_form_complete :
    NothingEscapes (Rec TMSAT) ∧
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ ¬ SATinP) ∧
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ Nonempty (LeastEscape TMSAT)) ∧
    (SelfRecursive TMSAT ↔ ¬ SATinP) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → Q → (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT)) ∧
    (RAcomp TMSAT ↔ (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT)) ∧
    (¬ ∃ b, comp b = b) :=
  ⟨tmsat_record_has_least_escape, pnp_normal_form_turing, tmsat_bit_is_the_act, tmsat_self_recursion.1,
   tmsat_every_route_passes_the_bit, tmsat_posit_is_the_bit, locus_vanishes⟩

/-! ## XVIII · the coincidence form and the unique certificate -/

/-- Registration is idempotent: the record of the record is the record. -/
theorem rec_rec (M : Machine) : Rec (Rec M) = Rec M := rfl

/-- THE COINCIDENCE NORMAL FORM: a machine is separated exactly when its escape profile coincides with its
    record's, which is empty. -/
theorem coincidence_normal_form (M : Machine) (x₀ : M.Inst) :
    (∀ A c k, Escapes M A c k ↔ Escapes (Rec M) A c k) ↔ Sep M :=
  ⟨fun h ⟨A, c, k, he⟩ => bnot_ne (M.sat x₀) ((h A c k).mp he x₀).1,
   fun hs A c k => ⟨fun he => absurd ⟨A, c, k, he⟩ hs, fun he => (bnot_ne (M.sat x₀) (he x₀).1).elim⟩⟩

theorem tmsat_coincidence : (∀ A c k, Escapes TMSAT A c k ↔ Escapes (Rec TMSAT) A c k) ↔ ¬ SATinP :=
  (coincidence_normal_form TMSAT []).trans sep_tmsat_iff

/-- THE CERTIFICATE IS UNIQUE: any two least-escape certificates of the SAT machine are the same object, and any
    two proofs of SAT ∉ P are the same proof, by proof irrelevance; so every route delivers one object. -/
theorem certificate_unique : (∀ L₁ L₂ : LeastEscape TMSAT, L₁ = L₂) ∧
    (∀ h₁ h₂ : ¬ SATinP, h₁ = h₂) :=
  ⟨fun _ _ => rfl, fun _ _ => rfl⟩

/-! ## XIX · the defense on the Turing machine -/

/-- NOTHING PASSES AGAINST THE SEPARATION BUT AN ESCAPE (classical): a rejection of SAT ∉ P is a witness, a
    decider and a bound with every run clean. -/
```

```
theorem tmsat_rejection_is_a_witness : ¬ ¬ SATinP ↔ ∃ (D : Decider) (c k : Nat), Escapes TMSAT D c k :=
  (fun hnn => Classical.byContradiction fun hne => hnn (sep_tmsat_iff.mp hne),
   fun he hn => sep_tmsat_iff.mpr hn he)

/-- EVERY PAIR LANDS (classical): every decider at every bound either is stopped by some string or escapes. -/
theorem tmsat_every_pair_lands (D : Decider) (c k : Nat) :
    (∃ w, ¬ Clean TMSAT D c k w) ∨ Escapes TMSAT D c k :=
  Classical.byCases (fun he : Escapes TMSAT D c k => Or.inr he)
    (fun hne => Or.inl (Classical.byContradiction fun hno =>
      hne (fun w => Classical.byContradiction fun hw => hno ⟨w, hw⟩)))

/-- FORCES OR HAS A TWIN (classical): every property of machines either forces the closure or holds on some
    machine that escapes. -/
theorem forces_or_has_a_twin (P : Machine → Prop) : Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
  Classical.byCases (fun hF : Forces P => Or.inl hF)
    (fun hF => Or.inr (Classical.byContradiction fun hne =>
      hF (fun M hp => Classical.byContradiction fun hc => hne ⟨M, hp, hc⟩)))

/-- An escape refutes the closure, on no axiom. -/
theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (he : Escapes M A c k) : ¬ NothingEscapes M :=
  fun h => match h A c k with | ⟨x, hx⟩ => hx (he x)

/-- The act leaves no escape, on no axiom. -/
theorem act_leaves_no_escape (M : Machine) (h : NothingEscapes M) : ∀ A c k, ¬ Escapes M A c k :=
  fun A c k he => escape_refutes M A c k he h

/-- THE EXTENSION of a machine by one more algorithm, which answers the predicate itself in no time. -/
def extendM (M : Machine) : Machine :=
  ⟨Option M.Alg, M.Inst, M.size,
   fun A x => match A with | none => M.sat x | some a => M.out a x,
   fun A x => match A with | none => 0 | some a => M.time a x,
   M.sat⟩

/-- The added algorithm escapes at every bound. -/
theorem extend_escapes (M : Machine) (c k : Nat) : Escapes (extendM M) none c k :=
  fun _ => ⟨rfl, Nat.zero_le _⟩

/-- Every examination of the machine is an examination of its extension. -/
def examination_extend (M : Machine) (e : Examination M) : Examination (extendM M) :=
  ⟨some e.A, e.c, e.k, e.x, e.stops⟩

/-- Every examination of the machine is an examination of its record. -/
def examination_rec (M : Machine) (e : Examination M) : Examination (Rec M) :=
  ⟨e.A, e.c, e.k, e.x, fun hc => bnot_ne (M.sat e.x) hc.1⟩

/-- NO RECORD DECIDES: every stock of examinations of a machine with an instance, finite or infinite, is a stock
    of stops both on its extension, where an algorithm escapes, and on its record, where nothing escapes.
    Observed stops decide nothing about the machine. -/
theorem no_record_decides (M : Machine) (x₀ : M.Inst) (ι : Type) (stock : ι → Examination M) :
    (∃ A c k, Escapes (extendM M) A c k) ∧
    (∀ i, ¬ Clean (extendM M) (some (stock i).A) (stock i).c (stock i).k (stock i).x) ∧
    NothingEscapes (Rec M) ∧
    (∀ i, ¬ Clean (Rec M) (stock i).A (stock i).c (stock i).k (stock i).x) :=
  ⟨⟨none, 0, 0, extend_escapes M 0 0⟩, fun i => (examination_extend M (stock i)).stops,
   record_has_least_escape M x₀, fun i => (examination_rec M (stock i)).stops⟩

theorem tmsat_no_record_decides (ι : Type) (stock : ι → Examination TMSAT) :
    (∃ A c k, Escapes (extendM TMSAT) A c k) ∧
    (∀ i, ¬ Clean (extendM TMSAT) (some (stock i).A) (stock i).c (stock i).k (stock i).x) ∧
    NothingEscapes (Rec TMSAT) ∧
    (∀ i, ¬ Clean (Rec TMSAT) (stock i).A (stock i).c (stock i).k (stock i).x) :=
  no_record_decides TMSAT [] ι stock

/-! ## XX · executable checks: the model and the decoder compute, by `decide` -/

/-- A three-state machine that accepts exactly when the first input symbol is `one`. -/
def firstIsOne : TM :=
  { q := 3, s := 0,
    δ := fun _ a => if a.val = 2 then ⟨(1, by decide), a, Dir.R⟩ else ⟨(2, by decide), a, Dir.R⟩,
    start := ⟨0, by decide⟩, accept := ⟨1, by decide⟩, reject := ⟨2, by decide⟩ }

example : ¬ HaltsAt firstIsOne [Sym.one] 0 := by decide
example : HaltsAt firstIsOne [Sym.one] 1 := by decide
example : verdictAt firstIsOne [Sym.one] 1 = true := by decide
example : verdictAt firstIsOne [Sym.zero] 1 = false := by decide
example : HaltsAt firstIsOne [] 1 := by decide
example : verdictAt firstIsOne [Sym.one, Sym.zero] 5 = true := by decide
example : encode [⟨0, true⟩]] = [Sym.one, Sym.sep, Sym.one, Sym.sep] := by decide
example : decode [Sym.one, Sym.sep, Sym.one, Sym.sep] = some [⟨0, true⟩]] := by decide
example : decode [Sym.one, Sym.one] = none := by decide
example : satLang [] = true := by decide
example : satLang [Sym.sep] = false := by decide
example : satLang (encode [⟨0, true⟩]]) = true := by decide
example : satLang (encode [⟨0, true⟩], ⟨0, false⟩]]) = false := by decide
example : satLang (encode [⟨1, true⟩, ⟨0, false⟩], ⟨1, false⟩]]) = true := by decide

end PNP.TM

/-! # PART D · THE FRONTIER, FORMAL ALONE: the black-box floor and the white-box contrast
    Carried from PNP_Frontier_II.lean of the register (namespaces BLACKBOX and MISSING), pure mathematics on no
    axiom; no physics and no root enter. -/
namespace PNP.BLACKBOX

def has : List Nat → Nat → Bool
  | [], _ => false
  | y :: ys, k => Nat.beq y k || has ys k

def drop : List Nat → Nat → List Nat
  | [], _ => []
  | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
  | 0 => rfl
  | n + 1 => beq_self n

theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
  | [], _ => Nat.le_refl 0
  | y :: ys, k => by
    unfold drop
    cases Nat.beq y k with
    | true => exact Nat.le_succ_of_le (drop_le ys k)
    | false => exact Nat.succ_le_succ (drop_le ys k)

theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length + 1 ≤ l.length
  | [], _, h => Bool.noConfusion h
  | y :: ys, k, h => by
    unfold drop
    unfold has at h
    cases hy : Nat.beq y k with
    | true => exact Nat.succ_le_succ (drop_le ys k)
    | false =>
      rw [hy] at h
      exact Nat.succ_le_succ (drop_len ys k h)

theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
    has l x = true → has (drop l k) x = true
  | [], _, _, _, h => Bool.noConfusion h
  | y :: ys, k, x, hxk, h => by
    unfold drop
    unfold has at h
    cases hy : Nat.beq y k with
    | true =>
      cases hyx : Nat.beq y x with
      | true =>
        have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans (Nat.eq_of_beq_eq_true hy)
        rw [this, beq_self] at hxk; exact Bool.noConfusion hxk
      | false =>
        rw [hyx] at h
        exact has_drop ys k x hxk h
    | false =>
      show has (y :: drop ys k) x = true
      unfold has
      cases hyx : Nat.beq y x with
      | true => rfl
      | false =>
        rw [hyx] at h
        exact has_drop ys k x hxk h

theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
```

```
  cases hb : Nat.beq x K with
  | false => rfl
  | true =>
    have e : x = K := Nat.eq_of_beq_eq_true hb
    rw [e] at h; exact absurd h (Nat.lt_irrefl K)

/-- PIGEONHOLE, on no axiom: a list that contains every number below K has at least K entries. -/
theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤ l.length
  | 0, _, _ => Nat.zero_le _
  | K + 1, l, h =>
    Nat.le_trans
      (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
        has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx)))))
      (drop_len l K (h K (Nat.lt_succ_self K)))

/-- The run of a test-only procedure with a fixed list of queries: it answers true when some queried
    candidate is a witness. -/
def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f

theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has l x
  | [], _ => rfl
  | y :: ys, x => by
    show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
    rw [run_eq_has ys x]

/-- THE BLACK-BOX BOUND: a fixed list of tests that decides existence over K candidates for every candidate
    function tests at least K of them. -/
theorem black_box_bound (K : Nat) (path : List Nat)
    (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < K ∧ f x = true) :
    K ≤ path.length :=
  pigeon K path (fun x hx => by
    rw [← run_eq_has]
    exact (correct _).mpr ⟨x, hx, beq_self x⟩)

/-- NO SAVINGS: over 2^N candidates no such list is shorter than 2^N. -/
theorem black_box_no_savings (N : Nat) (path : List Nat)
    (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < 2 ^ N ∧ f x = true) :
    ¬ path.length < 2 ^ N :=
  Nat.not_lt_of_le (black_box_bound (2 ^ N) path correct)

end PNP.BLACKBOX

namespace PNP.MISSING

/-- The black-box adversary family of `black_box_bound`: the point functions. -/
def point (x : Nat) : Nat → Bool := fun y => Nat.beq y x

/-- The same family, handed over as descriptions. -/
structure Desc where
  secret : Nat

def evalD (d : Desc) : Nat → Bool := point d.secret

/-- The white-box reader: one step, read the description. -/
def reader (d : Desc) : Nat := d.secret
def readerTime (_ : Nat) : Nat := 1

/-- The black-box hard instances are white-box trivial. -/
theorem reader_solves : ∀ d : Desc, evalD d (reader d) = true :=
  fun d => PNP.BLACKBOX.beq_self d.secret

structure Model where
  Alg     : Type
  decides : Alg → Prop
  time    : Alg → Nat → Nat

def PolyBounded (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c

/-- THE MISSING OBJECT: every algorithm that decides the problem from its description runs in
    super-polynomial time. -/
def WhiteBoxBound (M : Model) : Prop :=
  ∀ A : M.Alg, M.decides A → ¬ PolyBounded (M.time A)

theorem reader_poly : PolyBounded readerTime :=
  (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))

/-- A coherent model where descriptions reveal the witness: the object fails. -/
def revealing : Model := (Unit, fun _ => ∀ d : Desc, evalD d (reader d) = true, fun _ => readerTime)

theorem fails_when_descriptions_reveal : ¬ WhiteBoxBound revealing :=
  fun h => h () reader_solves reader_poly

/-- The object is keyed: it fails where descriptions reveal and holds wherever no algorithm decides. -/
theorem missing_object_is_keyed :
    (∃ M : Model, ¬ WhiteBoxBound M) ∧ (∀ M : Model, (∀ A, ¬ M.decides A) → WhiteBoxBound M) :=
  ((revealing, fails_when_descriptions_reveal), fun _ h A hA => absurd hA (h A))

end PNP.MISSING

/-! # PART E · THE RESIDUE, PRINTED AS A TYPE -/
namespace PNP.RESIDUE
open PNP.NORMALFORM

/-- THE RESIDUE. Everything above is unconditional. What remains of the separation is this one hypothesis, the
    transfer of least escape from the record to the machine, which is the act: given it, the separation follows
    on no axiom. It is printed here as the type of a hypothesis and is assumed nowhere in this file. -/
theorem separation_of_the_act (M : Machine) (x₀ : M.Inst)
    (bit : NothingEscapes (Rec M) → NothingEscapes M) : Sep M :=
  sep_from_the_cargo_loop M x₀ bit

/-- The residue on SAT, in every algorithm model: the one implication, and nothing else, stands between the
    theorems of this file and the separation of SAT. -/
theorem sat_separation_of_the_act (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat)
    (bit : NothingEscapes (Rec (PNP.SAT.SAT Alg out time)) → NothingEscapes (PNP.SAT.SAT Alg out time)) :
    PNP.SAT.SepSAT Alg out time :=
  separation_of_the_act (PNP.SAT.SAT Alg out time) [] bit

/-- THE RESIDUE ON THE TURING MODEL: given the one implication on the SAT machine of the Turing model, no Turing
    machine decides SAT within any polynomial bound. SAT ∉ P is this hypothesis and nothing else. -/
theorem sat_not_in_P_of_the_act
    (bit : NothingEscapes (Rec PNP.TM.TMSAT) → NothingEscapes PNP.TM.TMSAT) : ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (separation_of_the_act PNP.TM.TMSAT [] bit)

/-- THE RESIDUE AS THE SENTENCE: with SAT ∈ NP named as a hypothesis, the one implication gives P ≠ NP. -/
theorem p_ne_np_of_residue (hNP : PNP.TM.InNP PNP.TM.satLang)
    (bit : NothingEscapes (Rec PNP.TM.TMSAT) → NothingEscapes PNP.TM.TMSAT) : PNP.TM.P_ne_NP :=
  PNP.TM.p_ne_np_of_not_inP hNP (sat_not_in_P_of_the_act bit)

/-- THE WALL, restated on the residue: the residue cannot be supplied by any property shared by every machine
    (keyless_forces_nothing), nor by any principle true of every record (no_record_principle_decides), nor by
    any property shared by every algorithm model of SAT itself (sat_keyless_forces_nothing). What supplies it is
    keyed to the machine: the act. -/
theorem residue_is_keyed :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, (∀ M, P (Rec M)) → ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
    (∀ P : Machine → Prop, PNP.SAT.SATKeyless P → ¬ PNP.SAT.SATForces P) :=
  (keyless_forces_nothing, no_record_principle_decides, PNP.SAT.sat_keyless_forces_nothing)

end PNP.RESIDUE

/-! ## cones, pinned: the build is silent, and any drift fails it -/
/-- info: 'PNP.NORMALFORM.record_has_least_escape' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_has_least_escape
/-- info: 'PNP.NORMALFORM.record_is_separated' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_is_separated
/-- info: 'PNP.NORMALFORM.sep_normal_form' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.sep_normal_form
/-- info: 'PNP.NORMALFORM.sep_from_the_cargo_loop' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.sep_from_the_cargo_loop
/-- info: 'PNP.NORMALFORM.bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.bit_is_the_act
/-- info: 'PNP.NORMALFORM.record_is_self_recursive' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_is_self_recursive
/-- info: 'PNP.NORMALFORM.self_recursion_at_machine' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.self_recursion_at_machine
/-- info: 'PNP.NORMALFORM.self_verifying_iff_true' depends on axioms:
```

```
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.self_verifying_iff_true
/-- info: 'PNP.NORMALFORM.bit_is_keyed' does not depend on any axioms
      -/
#guard_msgs in #print axioms PNP.NORMALFORM.bit_is_keyed
/-- info: 'PNP.NORMALFORM.record_self_recursion_does_not_transfer'
      does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_self_recursion_does_
      not_transfer
/-- info: 'PNP.NORMALFORM.record_decides_nothing' does not depend on
      any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_decides_nothing
/-- info: 'PNP.NORMALFORM.every_route_passes_the_bit' depends on
      axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.every_route_passes_the_bit
/-- info: 'PNP.NORMALFORM.keyless_forces_nothing' does not depend on
      any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.keyless_forces_nothing
/-- info: 'PNP.NORMALFORM.no_record_principle_decides' does not depend
      on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.no_record_principle_decides
/-- info: 'PNP.NORMALFORM.examination_enacts_stage1' does not depend
      on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.examination_enacts_stage1
/-- info: 'PNP.NORMALFORM.stage1_is_local' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.stage1_is_local
/-- info: 'PNP.NORMALFORM.the_posit_is_the_bit' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.the_posit_is_the_bit
/-- info: 'PNP.NORMALFORM.locus_vanishes' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.locus_vanishes
/-- info: 'PNP.NORMALFORM.seatless_lock' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.seatless_lock
/-- info: 'PNP.NORMALFORM.pnp_normal_form_complete' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.pnp_normal_form_complete
/-- info: 'PNP.NORMALFORM.two_rows_two_bits' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.two_rows_two_bits
/-- info: 'PNP.SAT.satB_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.satB_iff
/-- info: 'PNP.SAT.empty_satisfiable' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.empty_satisfiable
/-- info: 'PNP.SAT.sat_record_has_least_escape' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_record_has_least_escape
/-- info: 'PNP.SAT.sat_normal_form' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_normal_form
/-- info: 'PNP.SAT.sat_bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_bit_is_the_act
/-- info: 'PNP.SAT.sat_self_recursion' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_self_recursion
/-- info: 'PNP.SAT.sat_every_route_passes_the_bit' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_every_route_passes_the_bit
/-- info: 'PNP.SAT.sat_posit_is_the_bit' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_posit_is_the_bit
/-- info: 'PNP.SAT.sat_normal_form_complete' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_normal_form_complete
/-- info: 'PNP.SAT.sat_bit_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_bit_is_keyed
/-- info: 'PNP.SAT.sat_keyless_forces_nothing' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_keyless_forces_nothing
/-- info: 'PNP.SAT.sat_no_record_principle_decides' does not depend on
      any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_no_record_principle_decides
/-- info: 'PNP.SAT.sat_record_decides_nothing' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_record_decides_nothing
/-- info: 'PNP.SAT.sat_premise_trichotomy' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_premise_trichotomy
/-- info: 'PNP.TM.decode_encode' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.decode_encode
/-- info: 'PNP.TM.encode_injective' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.encode_injective
/-- info: 'PNP.TM.satLang_encode_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.satLang_encode_iff
/-- info: 'PNP.TM.haltsAt_of_le' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.haltsAt_of_le
/-- info: 'PNP.TM.verdict_of_le' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.verdict_of_le
/-- info: 'PNP.TM.time_halts' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.time_halts
/-- info: 'PNP.TM.time_least' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.time_least
/-- info: 'PNP.TM.sep_tmsat_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.sep_tmsat_iff
/-- info: 'PNP.TM.evalPoly_le_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.evalPoly_le_bound
/-- info: 'PNP.TM.SATinP_of_poly' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.SATinP_of_poly
/-- info: 'PNP.TM.p_ne_np_of_not_inP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.p_ne_np_of_not_inP
/-- info: 'PNP.TM.tmsat_record_has_least_escape' does not depend on
      any axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_record_has_least_escape
/-- info: 'PNP.TM.pnp_normal_form_turing' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.pnp_normal_form_turing
/-- info: 'PNP.TM.tmsat_bit_is_the_act' does not depend on any axioms
      -/
#guard_msgs in #print axioms PNP.TM.tmsat_bit_is_the_act
/-- info: 'PNP.TM.tmsat_self_recursion' does not depend on any axioms
      -/
#guard_msgs in #print axioms PNP.TM.tmsat_self_recursion
/-- info: 'PNP.TM.tmsat_every_route_passes_the_bit' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_every_route_passes_the_bit
/-- info: 'PNP.TM.tmsat_posit_is_the_bit' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_posit_is_the_bit
/-- info: 'PNP.TM.tmsat_normal_form_complete' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_normal_form_complete
/-- info: 'PNP.TM.tmsat_racomp_iff_not_inP' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_racomp_iff_not_inP
/-- info: 'PNP.TM.every_exit_is_the_posit' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.every_exit_is_the_posit
/-- info: 'PNP.TM.the_posit_is_every_exit' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.TM.the_posit_is_every_exit
/-- info: 'PNP.TM.rec_rec' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.rec_rec
/-- info: 'PNP.TM.coincidence_normal_form' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.TM.coincidence_normal_form
/-- info: 'PNP.TM.tmsat_coincidence' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_coincidence
/-- info: 'PNP.TM.certificate_unique' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.certificate_unique
/-- info: 'PNP.TM.tmsat_rejection_is_a_witness' depends on axioms:
      [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_rejection_is_a_witness
/-- info: 'PNP.TM.tmsat_every_pair_lands' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_every_pair_lands
/-- info: 'PNP.TM.forces_or_has_a_twin' depends on axioms: [propext,
      Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.forces_or_has_a_twin
/-- info: 'PNP.TM.escape_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.escape_refutes
/-- info: 'PNP.TM.act_leaves_no_escape' does not depend on any axioms
      -/
#guard_msgs in #print axioms PNP.TM.act_leaves_no_escape
/-- info: 'PNP.TM.extend_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.extend_escapes
/-- info: 'PNP.TM.no_record_decides' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.no_record_decides
/-- info: 'PNP.TM.tmsat_no_record_decides' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_no_record_decides
/-- info: 'PNP.TM.TMSAT' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.TMSAT
/-- info: 'PNP.TM.InP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.InP
/-- info: 'PNP.TM.InNP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.InNP
/-- info: 'PNP.TM.P_ne_NP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.P_ne_NP
/-- info: 'PNP.TM.SATinP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.SATinP
/-- info: 'PNP.BLACKBOX.pigeon' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.BLACKBOX.pigeon
/-- info: 'PNP.BLACKBOX.black_box_bound' does not depend on any axioms
      -/
#guard_msgs in #print axioms PNP.BLACKBOX.black_box_bound
/-- info: 'PNP.BLACKBOX.black_box_no_savings' does not depend on any
      axioms -/
#guard_msgs in #print axioms PNP.BLACKBOX.black_box_no_savings
/-- info: 'PNP.MISSING.reader_solves' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.reader_solves
/-- info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.reader_poly
/-- info: 'PNP.MISSING.fails_when_descriptions_reveal' does not depend
      on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.fails_when_descriptions_reveal
/-- info: 'PNP.MISSING.missing_object_is_keyed' does not depend on any
      axioms -/
```

```
#guard_msgs in #print axioms PNP.MISSING.missing_object_is_keyed
/-- info: 'PNP.RESIDUE.separation_of_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.separation_of_the_act
/-- info: 'PNP.RESIDUE.sat_separation_of_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.sat_separation_of_the_act
/-- info: 'PNP.RESIDUE.sat_not_in_P_of_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.sat_not_in_P_of_the_act
/-- info: 'PNP.RESIDUE.p_ne_np_of_residue' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.p_ne_np_of_residue
/-- info: 'PNP.RESIDUE.residue_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.residue_is_keyed

/-! ============================================================================
    ============================================
    PART F · THE SUPPLY (namespace PNP.SUPPLY). The bit of Part E, supplied by the act and delivered by its two
    kinds: the formal cargo, and the physical witness. Each kind is a type whose inhabitant is the supply; neither
    is assumed anywhere in this file. Every exit is the posit (Part C), so each kind is the one bit in its own
    register, and either delivers SAT ∉ P and, with SAT ∈ NP, P ≠ NP.
================================================================================
    ============================================ -/
namespace PNP.SUPPLY
open PNP.NORMALFORM

/-- THE FORMAL KIND, the cargo: the record's least escape carried to the Turing SAT machine. -/
def FormalCargo : Prop := NothingEscapes (Rec PNP.TM.TMSAT) → NothingEscapes PNP.TM.TMSAT

/-- THE PHYSICAL KIND, the witness: every run of a decider is a physical process that registers each step at
    the Landauer floor, so its heat, counted in floor units, is its step count; and no decider answers every
    formula correctly within a polynomial heat budget. The second field is the one bit read in the register of
    heat; it is a premise of the witness, carried as a field, never proved. -/
structure PhysicalWitness where
  floor : Nat
  floor_pos : 0 < floor
  heat : PNP.TM.Decider → List PNP.TM.Sym → Nat
  priced : ∀ D w, heat D w = PNP.TM.time D w
  beyond_every_budget : ∀ (D : PNP.TM.Decider) (c k : Nat), ∃ w : List PNP.TM.Sym,
    PNP.TM.out D w ≠ PNP.TM.satLang w ∨ bound c k w.length < heat D w

/-- The cargo arrives with no premise: the record of the Turing SAT machine has least escape. -/
theorem cargo_arrives : NothingEscapes (Rec PNP.TM.TMSAT) :=
  PNP.TM.tmsat_record_has_least_escape

/-- KIND 1 DELIVERS: the formal cargo gives SAT ∉ P in one step. -/
theorem formal_kind_delivers (h : FormalCargo) : ¬ PNP.TM.SATinP :=
  PNP.RESIDUE.sat_not_in_P_of_the_act h

/-- The physical witness closes the Turing SAT machine: every decider at every bound is stopped. -/
theorem witness_closes (W : PhysicalWitness) : NothingEscapes PNP.TM.TMSAT :=
  fun D c k => match W.beyond_every_budget D c k with
    | ⟨w, Or.inl hne⟩ => ⟨w, fun hc => hne hc.1⟩
    | ⟨w, Or.inr hlt⟩ => ⟨w, fun hc =>
        Nat.lt_irrefl _ (Nat.lt_of_lt_of_le hlt (W.priced D w ▸ hc.2))⟩

/-- KIND 2 DELIVERS: the physical witness gives SAT ∉ P. -/
theorem physical_kind_delivers (W : PhysicalWitness) : ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (sep_of_closure PNP.TM.TMSAT (witness_closes W))

/-- THE TWO KINDS ARE ONE BIT: the physical witness is a formal cargo, and the cargo is exactly SAT ∉ P. -/
theorem the_two_kinds_are_one_bit :
    (PhysicalWitness → FormalCargo) ∧ (FormalCargo ↔ ¬ PNP.TM.SATinP) :=
  ⟨fun W _ => witness_closes W, PNP.TM.pnp_normal_form_turing⟩

/-- P ≠ NP, DELIVERED TWICE: with SAT ∈ NP, either kind gives the sentence. -/
theorem p_ne_np_delivered_twice (hNP : PNP.TM.InNP PNP.TM.satLang) :
    (FormalCargo → PNP.TM.P_ne_NP) ∧ (PhysicalWitness → PNP.TM.P_ne_NP) :=
  ⟨fun h => PNP.TM.p_ne_np_of_not_inP hNP (formal_kind_delivers h),
   fun W => PNP.TM.p_ne_np_of_not_inP hNP (physical_kind_delivers W)⟩

/-- THE ONE REFUTER: an escape, a decider and a bound with every run clean, refutes both kinds at once. -/
theorem the_one_refuter (e : ∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) :
    ¬ FormalCargo ∧ ¬ Nonempty PhysicalWitness :=
  have hs : ¬ Sep PNP.TM.TMSAT := fun hsep => hsep e
  ⟨fun h => hs (sep_of_closure _ (h cargo_arrives)),
   fun ⟨W⟩ => hs (sep_of_closure _ (witness_closes W))⟩

/-- THE SUPPLY, CLOSED. Nothing before the colon: the cargo arrives free; each kind delivers; the two kinds are
    one bit; the bit is SAT ∉ P; with SAT ∈ NP each kind gives P ≠ NP; and one escape refutes both. -/
theorem the_supply_closed :
    NothingEscapes (Rec PNP.TM.TMSAT) ∧
    (FormalCargo → ¬ PNP.TM.SATinP) ∧ (PhysicalWitness → ¬ PNP.TM.SATinP) ∧
    (PhysicalWitness → FormalCargo) ∧ (FormalCargo ↔ ¬ PNP.TM.SATinP) ∧
    (PNP.TM.InNP PNP.TM.satLang → (FormalCargo → PNP.TM.P_ne_NP) ∧ (PhysicalWitness → PNP.TM.P_ne_NP)) ∧
    ((∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) →
      ¬ FormalCargo ∧ ¬ Nonempty PhysicalWitness) :=
  ⟨cargo_arrives, formal_kind_delivers, physical_kind_delivers, the_two_kinds_are_one_bit.1,
   the_two_kinds_are_one_bit.2, p_ne_np_delivered_twice, the_one_refuter⟩

end PNP.SUPPLY

/-- info: 'PNP.SUPPLY.cargo_arrives' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.cargo_arrives
/-- info: 'PNP.SUPPLY.formal_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.formal_kind_delivers
/-- info: 'PNP.SUPPLY.witness_closes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.witness_closes
/-- info: 'PNP.SUPPLY.physical_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.physical_kind_delivers
/-- info: 'PNP.SUPPLY.the_two_kinds_are_one_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SUPPLY.the_two_kinds_are_one_bit
/-- info: 'PNP.SUPPLY.p_ne_np_delivered_twice' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.p_ne_np_delivered_twice
/-- info: 'PNP.SUPPLY.the_one_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.the_one_refuter
/-- info: 'PNP.SUPPLY.the_supply_closed' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SUPPLY.the_supply_closed
```

B. The compile receipt

Lean 4.19.0 (commit 6caaee842e94), `lean -DwarningAsError=true PNP_Normal_Form_Paper_v1_3_0_CC.lean`, exit code 0, nothing on standard output, nothing on standard error; the fourteen executable checks and the 93 `#guard_msgs` pins are closed inside this compile and print nothing. The 93 pinned lines, as they stand in the file, each the exact text the compiler produced for `#print axioms` on that constant:

```
'PNP.NORMALFORM.record_has_least_escape' does not depend on any axioms
'PNP.NORMALFORM.record_is_separated' does not depend on any axioms
'PNP.NORMALFORM.sep_normal_form' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.sep_from_the_cargo_loop' does not depend on any axioms
'PNP.NORMALFORM.bit_is_the_act' does not depend on any axioms
'PNP.NORMALFORM.record_is_self_recursive' does not depend on any axioms
'PNP.NORMALFORM.self_recursion_at_machine' does not depend on any axioms
'PNP.NORMALFORM.self_verifying_iff_true' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.bit_is_keyed' does not depend on any axioms
'PNP.NORMALFORM.record_self_recursion_does_not_transfer' does not depend on
  any axioms
'PNP.NORMALFORM.record_decides_nothing' does not depend on any axioms
'PNP.NORMALFORM.every_route_passes_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.keyless_forces_nothing' does not depend on any axioms
'PNP.NORMALFORM.no_record_principle_decides' does not depend on any axioms
'PNP.NORMALFORM.examination_enacts_stage1' does not depend on any axioms
'PNP.NORMALFORM.stage1_is_local' does not depend on any axioms
'PNP.NORMALFORM.the_posit_is_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.locus_vanishes' does not depend on any axioms
'PNP.NORMALFORM.seatless_lock' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.pnp_normal_form_complete' depends on axioms: [propext,
```

```
  Classical.choice, Quot.sound]
'PNP.NORMALFORM.two_rows_two_bits' does not depend on any axioms
'PNP.SAT.satB_iff' does not depend on any axioms
'PNP.SAT.empty_satisfiable' does not depend on any axioms
'PNP.SAT.sat_record_has_least_escape' does not depend on any axioms
'PNP.SAT.sat_normal_form' depends on axioms: [propext, Classical.choice,
  Quot.sound]
'PNP.SAT.sat_bit_is_the_act' does not depend on any axioms
'PNP.SAT.sat_self_recursion' does not depend on any axioms
'PNP.SAT.sat_every_route_passes_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.SAT.sat_posit_is_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.SAT.sat_normal_form_complete' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.SAT.sat_bit_is_keyed' does not depend on any axioms
'PNP.SAT.sat_keyless_forces_nothing' does not depend on any axioms
'PNP.SAT.sat_no_record_principle_decides' does not depend on any axioms
'PNP.SAT.sat_record_decides_nothing' does not depend on any axioms
'PNP.SAT.sat_premise_trichotomy' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.decode_encode' does not depend on any axioms
'PNP.TM.encode_injective' does not depend on any axioms
'PNP.TM.satLang_encode_iff' does not depend on any axioms
'PNP.TM.haltsAt_of_le' does not depend on any axioms
'PNP.TM.verdict_of_le' does not depend on any axioms
'PNP.TM.time_halts' does not depend on any axioms
'PNP.TM.time_least' does not depend on any axioms
'PNP.TM.sep_tmsat_iff' does not depend on any axioms
'PNP.TM.evalPoly_le_bound' does not depend on any axioms
'PNP.TM.SATinP_of_poly' does not depend on any axioms
'PNP.TM.p_ne_np_of_not_inP' does not depend on any axioms
'PNP.TM.tmsat_record_has_least_escape' does not depend on any axioms
'PNP.TM.pnp_normal_form_turing' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.tmsat_bit_is_the_act' does not depend on any axioms
'PNP.TM.tmsat_self_recursion' does not depend on any axioms
'PNP.TM.tmsat_every_route_passes_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.tmsat_posit_is_the_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.tmsat_normal_form_complete' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.tmsat_racomp_iff_not_inP' does not depend on any axioms
'PNP.TM.every_exit_is_the_posit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.the_posit_is_every_exit' does not depend on any axioms
'PNP.TM.rec_rec' does not depend on any axioms
'PNP.TM.coincidence_normal_form' does not depend on any axioms
'PNP.TM.tmsat_coincidence' does not depend on any axioms
'PNP.TM.certificate_unique' does not depend on any axioms
'PNP.TM.tmsat_rejection_is_a_witness' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.tmsat_every_pair_lands' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.forces_or_has_a_twin' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.TM.escape_refutes' does not depend on any axioms
'PNP.TM.act_leaves_no_escape' does not depend on any axioms
'PNP.TM.extend_escapes' does not depend on any axioms
'PNP.TM.no_record_decides' does not depend on any axioms
'PNP.TM.tmsat_no_record_decides' does not depend on any axioms
'PNP.TM.TMSAT' does not depend on any axioms
'PNP.TM.InP' does not depend on any axioms
'PNP.TM.InNP' does not depend on any axioms
'PNP.TM.P_ne_NP' does not depend on any axioms
'PNP.TM.SATinP' does not depend on any axioms
'PNP.BLACKBOX.pigeon' does not depend on any axioms
'PNP.BLACKBOX.black_box_bound' does not depend on any axioms
'PNP.BLACKBOX.black_box_no_savings' does not depend on any axioms
'PNP.MISSING.reader_solves' does not depend on any axioms
'PNP.MISSING.reader_poly' does not depend on any axioms
'PNP.MISSING.fails_when_descriptions_reveal' does not depend on any axioms
'PNP.MISSING.missing_object_is_keyed' does not depend on any axioms
'PNP.RESIDUE.separation_of_the_act' does not depend on any axioms
'PNP.RESIDUE.sat_separation_of_the_act' does not depend on any axioms
'PNP.RESIDUE.sat_not_in_P_of_the_act' does not depend on any axioms
'PNP.RESIDUE.p_ne_np_of_residue' does not depend on any axioms
'PNP.RESIDUE.residue_is_keyed' does not depend on any axioms
'PNP.SUPPLY.cargo_arrives' does not depend on any axioms
'PNP.SUPPLY.formal_kind_delivers' does not depend on any axioms
'PNP.SUPPLY.witness_closes' does not depend on any axioms
'PNP.SUPPLY.physical_kind_delivers' does not depend on any axioms
'PNP.SUPPLY.the_two_kinds_are_one_bit' depends on axioms: [propext,
  Classical.choice, Quot.sound]
'PNP.SUPPLY.p_ne_np_delivered_twice' does not depend on any axioms
'PNP.SUPPLY.the_one_refuter' does not depend on any axioms
'PNP.SUPPLY.the_supply_closed' depends on axioms: [propext,
  Classical.choice, Quot.sound]
```

Six controls of record, each a copy of the file with one line changed, each refused by the compiler with exit code 1. First, a theorem planted at the foot of Part E asserting ¬ SATinP with no hypothesis by passing the identity map as the transfer, so that the record's closure is offered where the machine's is required: refused with a type mismatch between `NothingEscapes (Rec TMSAT)` and `NothingEscapes TMSAT`. That refusal is the bit. Second, the executable check on the string `#` altered to claim it is in SAT: refused with *tactic `decide` proved that the proposition `satLang [Sym.sep] = true` is false*. Third, a two-state machine that accepts everything in zero steps, planted as a witness of SATinP: refused because `verdictAt alwaysAccept w 0 = satLang w` is not provable by reflexivity, as it is false on every unsatisfiable string. Fourth, the pin of `sep_normal_form` altered to read *does not depend on any axioms*: refused with *Docstring on `#guard_msgs` does not match generated message*, the compiler printing the true cone. Fifth, the sentence **P** ≠ **NP** planted as a theorem with `sorry` in place of both its hypotheses: a warning, *declaration uses 'sorry'*, which the flag makes an error. Sixth, the pin of `the_two_kinds_are_one_bit` in Part F altered to read *does not depend on any axioms*: refused with the same `#guard_msgs` mismatch, the compiler printing the true cone.

Author's Provenance and Method Disclosure

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below. The kernel of Appendix A carries the register's `_CC` mark: it was coded and compiled by the author's AI scribe, Claude, under the author's direction and specification, and its correctness rests on the Lean 4 kernel alone, which checked every theorem, closed every executable check and matched every pinned cone reproduced in Appendix B.

References

1. Cook, S. A. 1971. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.
2. Levin, L. A. 1973. Universal sequential search problems. *Problems of Information Transmission* 9(3): 265–266.
3. Karp, R. M. 1972. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, eds., Plenum, 85–103.
4. Baker, T., Gill, J., and Solovay, R. 1975. Relativizations of the P =? NP question. *SIAM Journal on Computing* 4(4): 431–442.
5. Razborov, A. A., and Rudich, S. 1997. Natural proofs. *Journal of Computer and System Sciences*

55(1): 24–35.
6. Aaronson, S., and Wigderson, A. 2009. Algebrization: a new barrier in complexity theory. *ACM Transactions on Computation Theory* 1(1): 2:1–2:54.
7. Williams, R. 2014. Nonuniform ACC circuit lower bounds. *Journal of the ACM* 61(1): 2:1–2:32.
8. Fortnow, L. 2009. The status of the P versus NP problem. *Communications of the ACM* 52(9): 78–86.
9. Impagliazzo, R. 1995. A personal view of average-case complexity. *Proceedings of the Tenth Annual IEEE Structure in Complexity Theory Conference*, 134–147.
10. Sipser, M. 1992. The history and status of the P versus NP question. *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*, 603–618.
11. Arora, S., and Barak, B. 2009. *Computational Complexity: A Modern Approach*. Cambridge University Press.
12. de Moura, L., and Ullrich, S. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, Springer, 625–635.
13. Sipser, M. 2013. *Introduction to the Theory of Computation*, third edition. Cengage Learning.
14. Islam, M. F. 2026. The Formal Closure of Computational Separation: P ≠ NP Closed on Existence Itself, Read on Computation. Zenodo, version 3.2.1, 4 October 2026. DOI 10.5281/zenodo.23133632. Register `1000sapients/Trisduction`, `Publication Library/Mathematics/Complexity Theory/P versus NP/`, with the kernels `PNP_Fortification.lean`, `PNP_Least_Escape.lean`, `PNP_Frontier_II.lean` and `PNP_Root.lean`.
15. Islam, M. 2026. MathOS^{T}, Formal-Alone, edition 1.0.5, carrying the formal master `RH_Both_Sides_Formal_Master`, with the theorems `zeta_ra_loop`, `record_has_least_erasure`, `rh_from_the_cargo_loop`, `every_route_passes_the_bit` and `self_verifying_iff_true`. Register `1000sapients/Trisduction`, path `MathOS/MathOS_v1_0_5_Formal.md`, SHA-256 `f6cbfbaf1b89ddf7 b5a8fa00334f5c1f af206781f2125e95 2eeea8839f4ba0a0`.
16. Hartmanis, J., and Stearns, R. E. 1965. On the computational complexity of algorithms. *Transactions of the American Mathematical Society* 117: 285–306.
17. Cook, S. A., and Reckhow, R. A. 1979. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic* 44(1): 36–50.
18. Mulmuley, K. D., and Sohoni, M. 2001. Geometric complexity theory I: an approach to the P vs. NP and related problems. *SIAM Journal on Computing* 31(2): 496–526.
19. Islam, M. TRISDUCTION: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Triaxial Orthogonality, Twelve-Gate Closure, the Quaternionic Completion, the Root Axiom, and the Master Pre-Sealed Proposition Ledger. Zenodo, version 4, 19 June 2026. DOI 10.5281/zenodo.20757507. https://zenodo.org/records/20757507. Mirror: PhilArchive record ISLTTG, https://philpapers.org/rec/ISLTTG. Master reference, continuously updated at the same location.
20. Gäher, L., and Kunze, F. 2021. Mechanising complexity theory: the Cook–Levin theorem in Coq. *12th International Conference on Interactive Theorem Proving (ITP 2021)*, Leibniz International Proceedings in Informatics 193, 20:1–20:18.
21. Balbach, F. J. 2023. The Cook–Levin theorem. *Archive of Formal Proofs*, January 2023.
22. Landauer, R. 1961. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5(3): 183–191.
23. Bérut, A., Arakelyan, A., Petrosyan, A., Ciliberto, S., Dillenschneider, R., and Lutz, E. 2012. Experimental verification of Landauer's principle linking information and thermodynamics. *Nature* 483: 187–189.
24. lean-dojo. 2025. Formalization of the Millennium Problems in Lean 4: `Problems/PVersusNP/Millennium.lean`. Reservoir package `@lean-dojo/problems`, repository `https://github.com/lean-dojo/LeanMillenniumPrizeProblems`.

End of manuscript.